

US Patent & Trademark Office

Patent Public Search | Text View

United States Patent Application Publication

20250278203

Kind Code

A1

Publication Date

September 04, 2025

Inventor(s)

KANTANI; Tomoyuki

MEMORY SYSTEM

Abstract

According to one embodiment, a controller identifies a fourth storage location on which a second step program operation is executed last among storage locations of a block and determines whether a condition that a fifth storage location stores unreadable data and each of memory cells of a sixth storage location has a threshold voltage corresponding to an erased state, is satisfied. Among the storage locations, in response to completion of a first step program operation on the fifth storage location, the second step program operation on the fourth storage location has been executed, and the first step program operation on the sixth storage location is to be executed after completion of the second step program operation on the fifth storage location.

Inventors: KANTANI; Tomoyuki (Yokohama, JP)

Applicant: Kioxia Corporation (Tokyo, JP)

Family ID: 87931695

Assignee: Kioxia Corporation (Tokyo, JP)

Appl. No.: 19/198464

Filed: May 05, 2025

Foreign Application Priority Data

JP 2022-035409

Mar. 08, 2022

Related U.S. Application Data

parent US division 18746394 20240618 parent-grant-document US 12340096 child US 19198464
parent US division 17821960 20220824 parent-grant-document US 12056368 child US 18746394

Publication Classification

Int. Cl.: G06F3/06 (20060101)

U.S. Cl.:

CPC G06F3/0619 (20130101); **G06F3/0629** (20130101); **G06F3/0679** (20130101);

Background/Summary

CROSS-REFERENCE TO RELATED APPLICATIONS [0001] This application is a division of and claims benefit under 35 U.S.C. § 120 to U.S. application Ser. No. 18/746,394 filed Jun. 18, 2024, which is a division of and claims benefit under 35 U.S.C. § 120 to U.S. application Ser. No. 17/821,960 filed Aug. 24, 2022 (now U.S. Pat. No. 12,056,368 issued Aug. 6, 2024), and claims the benefit of priority under 35 U.S.C. § 119 from Japanese Patent Application No. 2022-035409 filed Mar. 8, 2022, the entire contents of each of which are incorporated herein by reference.

FIELD

[0002] Embodiments described herein relate generally to a technique for controlling a nonvolatile memory.

BACKGROUND

[0003] In recent years, memory systems that includes a nonvolatile memory are widely used. Such a memory system is used as a storage for various computing devices, such as computers, mobile devices, and in-vehicle devices.

[0004] For example, in a case where the memory system is incorporated in a mobile device having a removable battery, when the battery is removed from the mobile device, power supply to the memory system is suddenly shut off without any notice to the memory system.

[0005] In this case, a measure against unexpected power loss, which is caused by suddenly shutting off of power supplied to the memory system, needs to be taken.

Description

BRIEF DESCRIPTION OF THE DRAWINGS

[0006] FIG. 1 is a block diagram illustrating an example of a configuration of an information processing system that includes a memory system according to a first embodiment.

[0007] FIG. 2 is a block diagram illustrating an example of a configuration of a NAND flash memory included in the memory system according to the first embodiment.

[0008] FIG. 3 illustrates an example of a configuration of a block of the NAND flash memory included in the memory system according to the first embodiment.

[0009] FIG. 4 illustrates an example of a circuit configuration of the block of the NAND flash memory included in the memory system according to the first embodiment.

[0010] FIG. 5 illustrates a state of data in a block when unexpected power loss occurred during a data write operation and a state of data in the block when the memory system is restarted, in the memory system according to the first embodiment.

[0011] FIG. 6 illustrates an example of threshold voltage distributions of memory cells of the NAND flash memory included in the memory system according to the first embodiment.

[0012] FIG. 7 illustrates the procedure of a multi-step program operation executed in the memory system according to the first embodiment.

[0013] FIG. 8 illustrates a writing order in the multi-step program operations executed in the memory system according to the first embodiment.

[0014] FIG. 9 illustrates a first example of a threshold voltage distribution of memory cells

included in a storage location in the memory system according to the first embodiment.

[0015] FIG. **10** illustrates a second example of threshold voltage distributions of memory cells included in a storage location in the memory system according to the first embodiment.

[0016] FIG. **11** illustrates a third example of threshold voltage distributions of memory cells included in a storage location in the memory system according to the first embodiment.

[0017] FIG. **12** illustrates a fourth example of threshold voltage distributions of memory cells included in a storage location in the memory system according to the first embodiment.

[0018] FIG. **13** illustrates an example of data write operations according to a write command received from a host and a garbage collection operation, executed in the memory system according to the first embodiment.

[0019] FIG. **14** illustrates a relationship between foggy-fine program operations and an unexpected power loss determination process that are executed in a memory system according to a comparative example.

[0020] FIG. **15** illustrates a first example of a relationship between foggy-fine program operations and an unexpected power loss determination process that are executed in the memory system according to the first embodiment.

[0021] FIG. **16** illustrates a second example of the relationship between the foggy-fine program operations and the unexpected power loss determination process that are executed in the memory system according to the first embodiment.

[0022] FIG. **17** illustrates a third example of the relationship between the foggy-fine program operations and the unexpected power loss determination process that are executed in the memory system according to the first embodiment.

[0023] FIG. **18** illustrates a fourth example of the relationship between the foggy-fine program operations and the unexpected power loss determination process that are executed in the memory system according to the first embodiment.

[0024] FIG. **19** illustrates an example of a relationship between foggy-fine program operations, which are executed on the head word line of a write destination block, and the unexpected power loss determination process that are executed in the memory system according to the comparative example.

[0025] FIG. **20** illustrates a first example of a relationship between foggy-fine program operations, which are executed on the head word line of a write destination block, and the unexpected power loss determination process that are executed in the memory system according to the first embodiment.

[0026] FIG. **21** illustrates a second example of the relationship between the foggy-fine program operations, which are executed on the head word line of the write destination block, and the unexpected power loss determination process that are executed in the memory system according to the first embodiment.

[0027] FIG. **22** illustrates a third example of the relationship between the foggy-fine program operations, which are executed on the head word line of the write destination block, and the unexpected power loss determination process that are executed in the memory system according to the first embodiment.

[0028] FIG. **23** illustrates a fourth example of the relationship between the foggy-fine program operations, which are executed on the head word line of the write destination block, and the unexpected power loss determination process that are executed in the memory system according to the first embodiment.

[0029] FIG. **24** illustrates a first example of a relationship between foggy-fine program operations and the unexpected power loss determination process that are executed in the memory system according to the first embodiment in a case where each block includes only one string unit.

[0030] FIG. **25** illustrates a second example of the relationship between the foggy-fine program operations and the unexpected power loss determination process that are executed in the memory

system according to the first embodiment in a case where each block includes only one string unit.

[0031] FIG. **26** illustrates a third example of the relationship between the foggy-fine program operations and the unexpected power loss determination process that are executed in the memory system according to the first embodiment in a case where each block includes only one string unit.

[0032] FIG. **27** illustrates a fourth example of the relationship between the foggy-fine program operations and the unexpected power loss determination process that are executed in the memory system according to the first embodiment in a case where each block includes only one string unit.

[0033] FIG. **28** is a flowchart illustrating the procedure of the unexpected power loss determination process executed in the memory system according to the first embodiment.

[0034] FIG. **29** is a flowchart illustrating the procedure of the data write operation executed in the memory system according to the first embodiment.

[0035] FIG. **30** is a block diagram illustrating an example of a configuration of an information processing system that includes a memory system according to a second embodiment.

[0036] FIG. **31** illustrates a foggy program operation executed in the memory system according to the second embodiment.

[0037] FIG. **32** illustrates a fine program operation executed in the memory system according to the second embodiment.

[0038] FIG. **33** illustrates a foggy program operation executed in the memory system according to the second embodiment in a case where each block includes only one string unit.

[0039] FIG. **34** illustrates a fine program operation executed in the memory system according to the second embodiment in a case where each block includes only one string unit.

[0040] FIG. **35** is a flowchart illustrating the procedure of a process of updating and storing foggy program management information in response to completion of a foggy program operation, the process being executed in the memory system according to the second embodiment.

[0041] FIG. **36** is a flowchart illustrating the procedure of a process of updating and storing the foggy program management information before starting a foggy program operation, the process being executed in the memory system according to the second embodiment.

[0042] FIG. **37** is a flowchart illustrating the procedure of an unexpected power loss determination process executed in the memory system according to the second embodiment.

[0043] FIG. **38** illustrates an example of a configuration of an information processing system that includes a memory system according to a third embodiment.

[0044] FIG. **39** illustrates a process of storing foggy program management information to a nonvolatile memory, which is executed in the memory system according to the third embodiment.

[0045] FIG. **40** is a flowchart illustrating the procedure of a process of managing the foggy program management information, which is executed in the memory system according to the third embodiment.

[0046] FIG. **41** illustrates an additional write operation of transitioning the threshold voltage of a memory cell from a foggy programmed state to a fine programmed state, which is executed in a memory system according to a fourth embodiment.

[0047] FIG. **42** illustrates a state of each storage location of a block in which unexpected power loss occurred and a state of each storage location of the block after execution of the additional write operation, which are in the memory system according to the fourth embodiment.

[0048] FIG. **43** is a flowchart illustrating the procedure of the additional write operation executed in the memory system according to the fourth embodiment.

[0049] FIG. **44** illustrates a first example and a second example of threshold voltage distributions of memory cells included in a storage location in a memory system according to a fifth embodiment.

[0050] FIG. **45** illustrates a third example and a fourth example of the threshold voltage distributions of the memory cells included in the storage location in the memory system according to the fifth embodiment.

[0051] FIG. **46** is a flowchart illustrating the procedure of a foggy program operation executed in the memory system according to the fifth embodiment.

[0052] FIG. **47** is a flowchart illustrating the procedure of a fine program operation executed in the memory system according to the fifth embodiment.

[0053] FIG. **48** is a flowchart illustrating the procedure of an unexpected power loss determination process executed in the memory system according to the fifth embodiment.

DETAILED DESCRIPTION

[0054] Various embodiments will be described hereinafter with reference to accompanying drawings.

[0055] In general, according to one embodiment, a memory system comprises a nonvolatile memory that includes a plurality of blocks, each of the plurality of blocks being a unit of a data erase operation and including a plurality of storage locations, each of the plurality of storage locations including a plurality of memory cells, and a controller electrically connected to the nonvolatile memory and configured to execute a data write operation on a write destination block among the plurality of blocks by using a multi-step program operation that includes at least a first step program operation and a second step program operation. The controller is configured to execute the data write operation on the plurality of storage locations of the write destination block in an order in which: the first step program operation on a first storage location of the write destination block is executed; after completion of the first step program operation on the first storage location, the first step program operation on a second storage location of the write destination block is executed; in response to completion of the first step program operation on the second storage location, the second step program operation on the first storage location is executed; and after completion of the second step program operation on the first storage location, the first step program operation on a third storage location of the write destination block is executed. In response to resuming of power supply to the memory system after shutoff of power supply to the memory system, the controller identifies, in a first block among the plurality of blocks, a fourth storage location on which the second step program operation is executed last among the plurality of storage locations of the first block according to the order. The controller determines whether a first condition that (A) a fifth storage location of the first block stores unreadable data and (B) each of the plurality of memory cells of a sixth storage location of the first block has a threshold voltage that corresponds to an erased state, is satisfied. Among the plurality of storage locations of the first block, according to the order, the first step program operation on the fourth storage location has been executed; after completion of the first step program operation on the fourth storage location, the first step program operation on the fifth storage location has been executed; in response to completion of the first step program operation on the fifth storage location, the second step program operation on the fourth storage location has been executed; and the first step program operation on the sixth storage location is to be executed after completion of the second step program operation on the fourth storage location. When the first condition is satisfied, the controller determines the sixth storage location of the first block as a next write destination storage location.

First Embodiment

[0056] FIG. **1** is a block diagram illustrating an example of a configuration of an information processing system **1** that includes a memory system **3** according to a first embodiment. The memory system **3** according to the first embodiment is a storage device including a nonvolatile memory.

[0057] The information processing system **1** includes a host (host device) **2** and the memory system **3**. The host **2** and the memory system **3** are connectable through a bus **4**.

[0058] The host **2** is an information processing device. The host **2** is, for example, a personal computer, a server computer, a mobile device, or an in-vehicle device. The host **2** accesses the memory system **3**. Specifically, the host **2** transmits a write command, which is a command to write

data, to the memory system 3. The host 2 transmits also a read command, which is a command to read data, to the memory system 3.

[0059] The memory system 3 is a semiconductor storage device. The memory system 3 is, for example, a solid state drive (SSD) or a universal flash storage (UFS) device. The memory system 3 includes a nonvolatile memory. The memory system 3 writes data into the nonvolatile memory. The memory system 3 reads data from the nonvolatile memory.

[0060] The bus 4 conforms to, for example, PCI Express™ (PCIe™) standard or Mobile Industry Processor Interface™ (MIPI™) standard. The bus 4 is a communication path that connects the host 2 and the memory system 3. The bus 4 is used to transmit data and an input/output (I/O) command from the host 2 to the memory system 3 and transmit data and a response from the memory system 3 to the host 2. The I/O command is a command for writing or reading data to or from the nonvolatile memory. Such an I/O command includes a write command and a read command.

[0061] An internal configuration of the host 2 will then be described. The host 2 includes a processor 21 and a memory 22.

[0062] The processor 21 is, for example, a central processing unit (CPU). The processor 21 communicates with the memory system 3 through the bus 4. The processor 21 executes a software (host software) that is loaded from the memory system 3 or another storage device connected to the host 2, onto the memory 22. The host software includes an operating system, a file system, a device driver, and an application program.

[0063] The memory 22 is, for example, a volatile memory. The memory 22 is, for example, a dynamic random access memory (DRAM).

[0064] An internal configuration of the memory system 3 will then be described. The memory system 3 includes a controller 5 and a nonvolatile memory 6. An example of the nonvolatile memory 6 is a NAND flash memory. Hereinafter, the nonvolatile memory 6 will be referred to as a NAND memory 6.

[0065] The controller 5 is a memory controller. The controller 5 is, for example, a semiconductor device, such as a system-on-a-chip (SoC). The controller 5 is electrically connected to the NAND memory 6. The controller 5 executes data writing to the NAND memory 6 and data reading from the NAND memory 6. In addition, the controller 5 executes communication with the host 2 through the bus 4. A physical interface connecting the controller 5 and the NAND memory 6 conforms to, for example, a Toggle NAND flash interface or an open NAND flash interface (ONFI). The function of each unit in the controller 5 may be realized by a dedicated hardware, a processor executing programs, or a combination thereof.

[0066] The NAND memory 6 is, for example, a flash memory of a two-dimensional structure or a flash memory of a three-dimensional structure. The NAND memory 6 includes multiple blocks. Each of the blocks is a minimum unit of erasing data stored in the NAND memory 6, that is, a unit of a data erase operation.

[0067] The memory system 3 may include a dynamic random access memory (DRAM) 7. The DRAM 7 is a volatile memory. A part of the storage area of the DRAM 7 temporarily stores, for example, write data received from the host 2 or read data read from the NAND memory 6. In other words, the DRAM 7 may be used as a write buffer or a read buffer.

[0068] The memory system 3 may further include a power supply circuit 8. The power supply circuit 8 is a power supply control circuit. The power supply circuit 8 supplies, for example, each component of the memory system 3 with power that is supplied from the host 2.

[0069] An internal configuration of the controller 5 will then be described. The controller 5 includes a host interface (host I/F) 51, a CPU 52, a static RAM (SRAM) 53, a direct memory access controller (DMAC) 54, an error checking and correction (ECC) circuit 55, a DRAM interface (DRAM I/F) 56, and a NAND interface (NAND I/F) 57. These units are interconnected through an internal bus 50.

[0070] The host interface 51 is a hardware interface circuit. The host interface 51 executes

communication with the host **2**. The host interface **51**, for example, receives an I/O command from the host **2**.

[0071] The CPU **52** is a processor. The CPU **52** controls the host interface **51**, the SRAM **53**, the DMAC **54**, the ECC circuit **55**, the DRAM interface **56**, and the NAND interface **57**. The CPU **52** loads a control program (firmware) stored in the NAND memory **6** or a ROM (not shown), onto the SRAM **53**, and executes the firmware to perform various processes. The firmware may be loaded onto the DRAM **7**.

[0072] The CPU **52** functions as, for example, a flash translation layer (FTL) configured to execute management of data stored in the NAND memory **6** and management of blocks included in the NAND memory **6**. The management of data stored in the NAND memory **6** includes, for example, management of mapping information indicating a correspondence relationship between each logical address and each physical address. The logical address is an address used by the host **2** for accessing the memory system **3**. The logical address is, for example, a logical block address (LBA). The physical address is an address that indicates a location of a physical storage element included in the NAND memory **6**. The CPU **52** manages mapping between each logical address and each physical address by using a logical-to-physical translation table (L2P table) **71**. The management of blocks included in the NAND memory **6** includes, for example, garbage collection, management of defective blocks (bad blocks), and wear leveling.

[0073] The SRAM **53** is a volatile memory. The SRAM **53** is used as, for example, a work area for the CPU **52**. A part of the storage area of the SRAM **53** may be used as a write buffer or a read buffer.

[0074] The DMAC **54** is a circuit that executes direct memory access (DMA). The DMAC **54** executes data transfer between the memory **22** of the host **2** and the DRAM **7** or the SRAM **53**.

[0075] The ECC circuit **55** is a circuit that executes data encoding and data decoding. The ECC circuit **55** executes an encoding process when data is written into the NAND memory **6**. In the encoding process, the ECC circuit **55** adds an ECC parity, which is a redundant code, to the data to be written. The ECC circuit **55** executes a decoding process by using an ECC parity that is added to data read from the NAND memory **6**. In the decoding process, the ECC circuit **55** corrects an error in the read data. When succeeding in correcting the error in the data read from the NAND memory **6**, that is, when having corrected the error in the read data, the ECC circuit **55** sends an ECC pass notice to the CPU **52**. When failing in correcting the error in the data read from the NAND memory **6**, that is, when not having corrected the error in the read data, the ECC circuit **55** notifies the CPU **52** of an ECC error.

[0076] The DRAM interface **56** is a circuit that controls the DRAM **7**. The DRAM interface **56** stores data in the DRAM **7** and reads data stored in the DRAM **7**.

[0077] The NAND interface **57** is a circuit that controls the NAND memory **6**. The NAND interface **57** includes, for example, NAND controllers **571-0**, **571-1**, **571-2**, . . . , and **571-31**. Each of the NAND controllers **571-0**, **571-1**, **571-2**, . . . , and **571-31** is connected to one or more flash dies included in the NAND memory **6**, through a corresponding one channel of one or more channels (ch) **0**, **1**, **2**, . . . , and **31**. The flash die is referred to also as a flash chip. The NAND controllers **571-0**, **571-1**, **571-2**, . . . , and **571-31** control, for example, a flash die (**#0**) **61-0**, a flash die (**#1**) **61-1**, . . . , and a flash die (**#31**) **61-31**, respectively, which are included in the NAND memory **6**.

[0078] Information stored in the DRAM **7** will then be described. The information stored in the DRAM **7** includes the L2P table **71**, a block management table **72**, a free block list **73**, a single-level cell (SLC) active block list **74**, and a quad-level cell (QLC) active block list **75**. Some or all of the L2P table **71**, the block management table **72**, the free block list **73**, the SLC active block list **74**, and the QLC active block list **75** may be stored not in the DRAM **7** but in the SRAM **53**.

[0079] The L2P table **71** manages mapping between each logical address and each physical address with a certain size unit, such as a sector.

[0080] The block management table **72** manages each of blocks of the NAND memory **6**. The block management table **72** manages the blocks to indicate whether each block is available.

[0081] The free block list **73** manages each of blocks of the NAND memory **6** in which valid data is not stored, as a free block. Data stored in each block is roughly divided into valid data and invalid data. The valid data means the latest data to which the L2P table **71** refers. Specifically, the valid data is data stored in a physical storage element whose location is specified by a physical address that is associated with (i.e., mapped to) a logical address. The valid data is data to possibly be read by the host **2** later. On the other hand, the invalid data means old data to which the L2P table **71** no longer refers. Specifically, the invalid data is data stored in a physical storage element whose location is specified by a physical address that is not associated with (i.e., not mapped to) a logical address. The invalid data is data not to be read by the host **2** anymore.

[0082] The SLC active block list **74** and the QLC active block list **75** manage active blocks of the NAND memory **6**. The active block stores at least valid data. Specifically, the SLC active block list **74** manages active blocks each storing valid data written in an SLC mode. The SLC mode is a writing mode in which 1-bit data is stored in one memory cell of the NAND memory **6**. The QLC active block list **75** manages active blocks each storing valid data written in a QLC mode. The QLC mode is a writing mode in which 4-bit data is stored in one memory cell of the NAND memory **6**.

[0083] An example of a configuration of the flash die **61** included in the NAND memory **6** will then be described. FIG. **2** is a block diagram illustrating an example of the configuration of the flash die **61** of the NAND memory **6** included in the memory system **3** according to the first embodiment.

[0084] The configuration of the flash die **61** will be described with focusing on the flash die (#0) **61-0**. Other flash dies are the same in configuration as the flash die (#0) **61-0**. The flash die (#0) **61-0** includes a peripheral circuit **611-0** and a memory cell array **612-0**.

[0085] The peripheral circuit **611-0** controls the memory cell array **612-0**. The peripheral circuit **611-0** includes, for example, a row decoder, a column decoder, a sense amplifier, a latch circuit, and a voltage generation circuit. In response to reception of an address and a command from the NAND controller **571-0**, the peripheral circuit **611-0** executes a program operation, a read operation, or a data erase operation on the memory cell array **612-0**.

[0086] The memory cell array **612-0** includes a plurality of blocks (BLK0, BLK1, BLK2, . . .). Each block BLK is a set of nonvolatile memory cell transistors (which will hereinafter be simply referred to as memory cell transistors or memory cells). Each block includes a plurality of string units (SU0, SU1, SU2, and SU3). Each string unit SU is a set of memory cell transistors. Each string unit SU includes a plurality of NAND strings NS (which is also simply referred to as strings). Each NAND string NS is a set of memory cell transistors.

[0087] FIG. **2** illustrates a case where each block includes four string units SU0, SU1, SU2, and SU3. The number of string units SU included in each block may be three or less, or may be five or more. A configuration in which each block includes only one string unit SU may also be adopted. In short, each block includes one or more string units SU.

[0088] A configuration example of the block will then be described. FIG. **3** illustrates an example of the configuration of the block of the NAND memory **6** included in the memory system **3** according to the first embodiment.

[0089] In FIG. **3**, the configuration of the block will be described with focusing on the block BLK0. Other blocks are the same in configuration as the block BLK0. The block BLK0 includes four string units (SU0, SU1, SU2, and SU3). The four string units (SU0, SU1, SU2, and SU3) are arranged in a direction (horizontal direction) orthogonal to a direction (vertical direction) in which a plurality of word lines WL0 to WL7 are stacked. Each string unit SU includes a plurality of NAND strings NS. One end of each NAND string NS is connected to the corresponding bit line among a plurality of bit lines (BLO to BL(L-1)). Each NAND string NS extends in the vertical direction. Control gates of the memory cell transistors included in each NAND string NS are

connected to the word lines (WL0, WL1, . . . , WL7), respectively.

[0090] A circuit configuration of the block will then be described. FIG. 4 illustrates an example of the circuit configuration of the block of the NAND memory 6 included in the memory system 3 according to the first embodiment.

[0091] In FIG. 4, the configuration of the block will be described with focusing on the block BLK0. Other blocks are the same in configuration as the block BLK0. The block BLK0 includes four string units (SU0, SU1, SU2, and SU3). Each string unit SU includes a plurality of NAND strings NS.

[0092] Each NAND string NS includes, for example, eight memory cell transistors MT (MT0 to MT7) and two selection transistors ST1 and ST2. Each memory cell transistor MT includes a control gate and a charge accumulation layer, and stores data in a nonvolatile manner. The memory cell transistors MT (MT0 to MT7) are connected in series between the source of the selection transistor ST1 and the drain of the selection transistor ST2. The number of memory cell transistors MT in the NAND string NS is not limited to eight.

[0093] Gates of respective selection transistors ST1 of the string unit SU0 are connected to a selection gate line SGD0 that corresponds to the string unit SU0. Gates of respective selection transistors ST1 of the string unit SU1 are connected to a selection gate line SGD1 that corresponds to the string unit SU1. Gates of respective selection transistors ST1 of the string unit SU2 are connected to a selection gate line SGD2 that corresponds to the string unit SU2. Gates of respective selection transistors ST1 of the string unit SU3 are connected to a selection gate line SGD3 that corresponds to the string unit SU3. In contrast, gates of selection transistors ST2 of the string units SU0 to SU3 are all connected commonly to a selection gate line SGS. The gates of the selection transistors ST2 of the string unit SU0 to the string unit SU3, however, may be connected respectively to a selection gate line (SGS0) to a selection gate line (SGS3), which are different from each other. Control gates of the memory cell transistors MT0 to MT7 in the same block BLK are commonly connected to the word lines WL0 to WL7, respectively.

[0094] A program operation and a read operation by the peripheral circuit 611 can each be collectively executed on a plurality of memory cells MT that are connected to one word line in one string unit SU. A set of memory cells MT that are selected collectively at execution of the program operation or the read operation is referred to as a memory cell group MG. The memory cell group MG is a unit of the program operation and of the read operation, and is treated as a storage location. The size of data that is stored in each memory cell group MG in a case where each memory cell MT is configured to store 1-bit data is referred to as a page. In a case where each memory cell MT is configured to store 4-bit data, the size of data stored in each memory cell group MG is 4 pages.

[0095] A data erase operation by the peripheral circuit 611 is executed in units of blocks. In other words, the data, which is stored in every memory cell MT included in one block, is erased collectively.

[0096] A restart process that is executed after unexpected power loss occurred during a data write operation will be described. FIG. 5 illustrates a state of data in the block BLK0 when unexpected power loss occurred during a data write operation, and a state of data in the block BLK0 when the memory system 3 is restarted, in the memory system 3 according to the first embodiment.

Hereinafter, a case where a data write operation is executed by using a full sequence program operation will be described. The full sequence program operation is a program operation by which data is written into one storage location (memory cell group MG) in one string unit SU at a time. The program operation includes, for example, one or more program loops. Each program loop includes a first operation of injecting charges into the charge accumulation layer of the memory cell MT to increase a threshold voltage of the memory cell MT, and a second operation of verifying whether the threshold voltage of the memory cell MT is equal to or higher than a voltage corresponding to data to be written.

[0097] (a) The controller 5 executes a data write operation on the block BLK0, which is allocated as a write destination block, using the full sequence program operation. Data stored in each storage location where the program operation is completed is referred to as write-completed data. The write-completed data is readable data. The controller 5 periodically records, in the NAND memory 6, information indicating a storage location at which the data write operation is to be executed next among storage locations included in the write destination block. The information is recorded as a write destination storage location. In FIG. 5, the controller 5 records a write destination location A as the storage location at which the data write operation is to be executed next.

[0098] (b) Subsequently, the controller 5 executes the data write operation using the full sequence program operation, from the storage location indicated by the write destination location A to a storage location indicated by a write destination location B. Now a case is assumed where unexpected power loss occurred when the program operation is being executed on the storage location indicated by the write destination location B. Unexpected power loss is an event that power supply to the memory system 3 is shut off without receiving a shutdown notice from the host 2 to the memory system 3 in advance.

[0099] (c) When power loss occurred when the program operation is being executed on the storage location indicated by the write destination location B, the program operation is interrupted before being completed. As a result, this storage location becomes a storage location including unreadable data. Unreadable data is data that cannot be correctly read from the NAND memory 6. When data is read from a storage location that includes unreadable data, the ECC circuit 55 cannot correct an error in the data, and therefore outputs a status indicating an ECC error. In some cases, the threshold voltage of each memory cell MT included in the storage location where the unreadable data is stored is already changed by the interrupted program operation. For this reason, the controller 5 does not re-execute the program operation on this storage location.

[0100] (d) When power supply to the memory system 3 is resumed and the memory system 3 restarts, the controller 5 acquires information indicating a write destination storage location, from the NAND memory 6. Here, information indicating the write destination location B is not recorded in the NAND memory 6. The information acquired from the NAND memory 6, therefore, indicates the write destination location A. The controller 5 reads data from the storage location indicated by the acquired write destination location A, and determines whether readable data is stored in the storage location. Readable data is data that can be correctly read from the NAND memory 6. When readable data is stored in this storage location, error correction by the ECC circuit 55 may be successful. In other words, the controller 5 can determine whether data stored in the NAND memory 6 is readable data or unreadable data on the basis of an ECC pass notice or an ECC error notice that is sent from the ECC circuit 55 to the CPU 52. When data stored in the storage location, which is indicated by the write destination location A, is readable data, the controller 5 further reads data stored in a storage location subsequent to the write destination location A, and determines whether data stored in this storage location is readable data.

[0101] (e) By repeating the operation described in (d), the controller 5 identifies the first storage location that stores unreadable data. The controller 5 then determines whether the identified storage location is in an erased state. On every memory cell MT included in the storage location in the erased state, no program operation is executed after a data erase operation. When the identified storage location is in the erased state, the controller 5 is capable of executing a data write operation on the identified storage location. Therefore, the controller 5 continues the data write operation on the block BLK0. On the other hand, when the identified storage location is not in the erased state, the controller 5 is incapable of continuing the data write operation on the identified storage location. In this case, the controller 5 may record the block BLK0 in the block management table 72, as a block unavailable for data writing, and newly allocate a different block as a write destination block. Then, the controller 5 may execute the data write operation on the block newly allocated as the write destination block.

[0102] In this manner, in the case of using the full sequence program operation, the controller 5 can determine whether power loss occurred during execution of the program operation, by performing the process of determining whether readable data is stored in each of several storage locations and the process of determining whether a storage location which is firstly identified to store unreadable data is in the erased state.

[0103] Threshold voltage distributions of a plurality of memory cells to which data is written in the QLC mode will then be described. In the QLC mode, 4-bit data is written to one memory cell. FIG. 6 illustrates an example of threshold voltage distributions of a plurality of memory cells of the NAND memory 6 included in the memory system 3 according to the first embodiment.

[0104] The threshold voltage V_{th} of each memory cell changes depending on the amount of charge held by the charge accumulation layer of each memory cell. The NAND memory 6 stores information by mapping a plurality of data values, which represents the information to be stored, with threshold voltage distributions of the threshold voltage V_{th} of the memory cell.

[0105] In the QLC mode, the threshold voltage each memory cell may take is divided into 16 threshold voltage ranges. These 16 threshold voltage ranges are referred to as an Er state, an A state, a B state, a C state, . . . , and an O state in the ascending order of the threshold voltages. The threshold voltage V_{th} of each memory cell is controlled by the program operation such that the threshold voltage V_{th} belongs to any one of the threshold voltage ranges.

[0106] These 16 states correspond respectively to different pieces of 4-bit data. The threshold voltage V_{th} of a memory cell in which 4-bit data corresponding to the A state is programmed is set in a threshold voltage range between V_{r1} to V_{r2} . Likewise, the threshold voltage V_{th} of a memory cell in which 4-bit data corresponding to the B state is programmed is set in a threshold voltage range between V_{r2} to V_{r3} . Likewise, the threshold voltage V_{th} of a memory cell in which 4-bit data corresponding to the O state is programmed is set in a threshold voltage range between V_{r15} to V_{r16} . This pattern of setting the threshold voltage V_{th} applies also to the other states.

[0107] A multi-step program operation will then be described. FIG. 7 illustrates the procedure of the multi-step program operation that is executed in the memory system 3 according to the first embodiment when data is written in the QLC mode.

[0108] The controller 5 executes a data write operation on each of a plurality of storage locations included in a write destination block by using multi-step program operation that includes at least a first step program operation and a second step program operation. In the first step program operation, data of four pages to be written is transferred to the NAND memory 6, and the data is written into a plurality of memory cells (memory cell group MG) that are connected to a certain word line and are included in a certain storage location in the write destination block. In the first step program operation, the threshold voltage of each memory cell is set with low precision. In this state, data cannot be read correctly from each memory cell. In the second step program operation, the same data of four pages as the data transferred in the first step program operation is transferred to the NAND memory 6 again, and the data is written into the memory cell group MG corresponding to the storage location. In the second step program operation, the threshold voltage of each memory cell is set to a target threshold voltage that corresponds to the data to be written. When the second step program operation is completed, data can be read correctly from each memory cell.

[0109] The second step program operation on each storage location is executed after the first step program operation on a different storage location is completed. The different storage location is connected to a word line subsequent to the word line that connects the storage location to which the second step program operation is to be executed.

[0110] As multi-step program operation, for example, a foggy-fine program operation may be used. The foggy-fine program operation includes a foggy program operation and a fine program operation. The foggy program operation is a program operation (the first step program operation) by which the threshold voltage of each memory cell is set with low precision. The fine program

operation is a program operation (the second step program operation) by which the threshold voltage of each memory cell is adjusted to set it to the target threshold voltage.

[0111] Foggy-fine program operations on a write destination block are executed in the following procedure. Hereinafter, a storage location (i.e., memory cell group MG) that is connected to a certain word line WLx and is included in a certain string unit SUy will be referred to as an MG (WLx, SUy). [0112] (1) First, a foggy program operation is executed on an MG (WL0, SU0). At this point, write data having the size of four pages, that is, write data having a size that is four times as large as a page size is transferred to the NAND memory 6. The page size is equivalent to the number of strings included in one string unit SU, that is, the number of memory cells included in one storage location (i.e., memory cell group MG). [0113] (2) Subsequently, a foggy program operation is executed on an MG (WL0, SU1). [0114] (3) Subsequently, a foggy program operation is executed on an MG (WL0, SU2). [0115] (4) Subsequently, a foggy program operation is executed on an MG (WL0, SU3). [0116] (5) When the foggy program operations on the MG (WL0, SU0) to the MG (WL0, SU3) are completed, a foggy program operation on an MG (WL1, SU0) is executed. [0117] (6) When the foggy program operation on the MG (WL1, SU0) is completed, a fine program operation on the MG (WL0, SU0) is executed. At this point, the same write data as the write data used in the foggy program operation on the MG (WL0, SU0) is transferred to the NAND memory 6 again. Then, the fine program operation is executed on the MG (WL0, SU0). When this fine program operation is completed, the MG (WL0, SU0) transitions to a fine programmed state. Data stored in a storage location in the fine programmed state is readable data that can be read correctly. [0118] (7) Subsequently, a foggy program operation is executed on an MG (WL1, SU1). [0119] (8) When the foggy program operation on the MG (WL1, SU1) is completed, a fine program operation on the MG (WL0, SU1) is executed. When the fine program operation is completed, the MG (WL0, SU1) transitions to the fine programmed state. [0120] (9) Subsequently, a foggy program operation is executed on an MG (WL1, SU2). [0121] (10) When the foggy program operation on the MG (WL1, SU2) is completed, a fine program operation on the MG (WL0, SU2) is executed. As a result, the MG (WL0, SU2) transitions to the fine programmed state. [0122] (11) Subsequently, a foggy program operation is executed on an MG (WL1, SU3). [0123] (12) When the foggy program operation on the MG (WL1, SU3) is completed, a fine program operation on the MG (WL0, SU3) is executed. As a result, the MG (WL0, SU3) transitions to the fine programmed state. [0124] (13) When the foggy program operations on the MG (WL1, SU0) to the MG (WL1, SU3) are completed, a foggy program operation on an MG (WL2, SU0) is executed. [0125] (14) When the foggy program operation on the MG (WL2, SU0) is completed, a fine program operation on the MG (WL1, SU0) is executed. As a result, the MG (WL1, SU0) transitions to the fine programmed state. [0126] (15) Subsequently, a foggy program operation is executed on an MG (WL2, SU1). [0127] (16) When the foggy program operation on the MG (WL2, SU1) is completed, a fine program operation on the MG (WL1, SU1) is executed. As a result, the MG (WL1, SU1) transitions to the fine programmed state. [0128] (17) Subsequently, a foggy program operation is executed on an MG (WL2, SU2). [0129] (18) When the foggy program operation on the MG (WL2, SU2) is completed, a fine program operation on the MG (WL1, SU2) is executed. As a result, the MG (WL1, SU2) transitions to the fine programmed state.

[0130] The order in which foggy-fine program operations are executed on each of the storage locations included in each block will then be described. FIG. 8 illustrates a writing order of multi-step program operations on a block, which is executed in the memory system 3 according to the first embodiment.

[0131] Each block includes one or more string units SU. The controller 5 executes foggy program operations on one or more memory cell groups MG (referred to as first storage locations) that are connected to a certain word line (first word line) and are included in one or more string units SU, respectively. After completing the foggy program operations on the one or more first storage

locations, the controller 5 executes a foggy program operation on one second storage location among one or more memory cell groups MG (referred to as second storage locations) that are connected to a second word line subsequent to the first word line and that are included in one or more string units SU, respectively.

[0132] Then, in response to completion of the foggy program operation on the one second storage location, the controller 5 executes a fine program operation on one of the first storage locations that is connected to the first word line and that is included in the same string unit as the one second storage location.

[0133] The controller 5 executes the foggy-fine program operations on storage locations included in the write destination block in such a writing order as described above. Data written to each storage location becomes readable after completion of the fine program operation on each storage location.

[0134] In FIG. 8, the order of data writing to a plurality of storage locations included in one block are denoted by numbers. FIG. 8 illustrates an assumed case where the number of word lines per block is eight (word lines WL0 to WL7). The foggy program operation on the MG (WL0, SU0) is executed as the first data write operation for the block.

[0135] The foggy program operation on the MG (WL0, SU1) is executed as the second data write operation for the block. The foggy program operation on the MG (WL0, SU2) is executed as the third data write operation for the block. The foggy program operation on the MG (WL0, SU3) is executed as the fourth data write operation for the block.

[0136] The foggy program operation on the MG (WL1, SU0) is executed as the fifth data write operation for the block. The fine program operation on the MG (WL0, SU0) is executed as the sixth data write operation for the block.

[0137] The foggy program operation on the MG (WL1, SU1) is executed as the seventh data write operation for the block. The fine program operation on the MG (WL0, SU1) is executed as the eighth data write operation for the block.

[0138] The foggy program operation on the MG (WL1, SU2) is executed as the ninth data write operation for the block. The fine program operation on the MG (WL0, SU2) is executed as the tenth data write operation for the block.

[0139] The foggy program operation on the MG (WL1, SU3) is executed as the 11th data write operation for the block. The fine program operation on the MG (WL0, SU3) is executed as the 12th data write operation for the block.

[0140] The foggy program operation on the MG (WL2, SU0) is executed as the 13th data write operation for the block. The fine program operation on the MG (WL1, SU0) is executed as the 14th data write operation for the block.

[0141] In this manner, an MG (WL x, SU y+1) is a storage location on which a foggy program operation is to be executed subsequently to execution of a foggy program operation on an MG (WL x, SU y). An MG (WL x+1, SU0) is a storage location on which a foggy program operation is to be executed subsequently to execution of a foggy program operation on an MG (WL x, SU3). An MG (WL x+1, SU y) is a storage location on which a foggy program operation has been executed previously to execution of a fine program operation on an MG (WL x, SU y). An MG (WL x+1, SU y+1) is a storage location on which a foggy program operation is to be executed subsequently to execution of a fine program operation on the MG (WL x, SU y).

[0142] Threshold voltage distributions of memory cells included in each storage location will then be described. Threshold voltages of memory cells included in a storage location in the erased state will first be described. FIG. 9 illustrates threshold voltage distributions of memory cells included in a storage location in the erased state in the memory system 3 according to the first embodiment.

[0143] The threshold voltage of each of the memory cells included in the storage location in the erased state is lower than an erased page threshold voltage. The erased page threshold voltage is, for example, 0 V. The erased page threshold voltage is, for example, Vr1 described with reference

to FIG. 6. In other words, the storage location in the erased state is a storage location including memory cells whose threshold voltages are all kept in the threshold voltage range of the Er state after execution of the data erase operation. The storage location in the erased state does not include any memory cell whose threshold voltage is equal to or higher than the erased page threshold voltage. Accordingly, the controller 5 performs an erased page check on a storage location, thereby determining whether the storage location is in the erased state. In the erased page check, the controller 5 determines whether the storage location includes a memory cell whose threshold voltage is equal to or higher than the erased page threshold voltage.

[0144] The storage location in the erased state is a storage location where data cannot be read correctly. When data is read from the storage location in the erased state, the ECC circuit 55 cannot correct an error in the read data since the data includes no ECC parity.

[0145] In the erased page check, the controller 5 may directly acquire raw data read from the storage location by bypassing the ECC circuit 55. When data from all memory cells included in the raw data is data corresponding to the Er state, the controller 5 determines that all the memory cells included in the storage location have the threshold voltage corresponding to the erased state.

[0146] When the controller 5 includes a circuit that counts the number of memory cells each having a threshold voltage equal to or higher than the erased page threshold voltage, the controller 5 may determine whether all the memory cells included in the storage location each have the threshold voltage corresponding to the erased state on the basis of a count value outputted from the circuit. This circuit may control the NAND memory 6 in such a way as to apply the erased page threshold voltage as a read voltage, to a word line of an erased page check target. This circuit may count the number of memory cells that store data, which is included in data read from the NAND memory 6, having different value from that corresponding to the Er state, as the number of memory cells each having a threshold voltage equal to or higher than the erased page threshold voltage.

[0147] The storage location in the erased state is a storage location on which a data write operation is executable.

[0148] Threshold voltage distributions of a plurality of memory cells included in a storage location where readable data is stored will then be described. FIG. 10 illustrates threshold voltage distributions of a plurality of memory cells included in a storage location where data is readable, in the memory system 3 according to the first embodiment.

[0149] A storage location where data is readable is a storage location in a full sequence programmed state in which a full sequence program operation has been completed, or a storage location in a fine programmed state in which a fine program operation has been completed. When data is read from this storage location, the ECC circuit 55 may correct an error in the read data since the data includes an ECC parity. The threshold voltage of each memory cell included in this storage location has been changed to a threshold voltage corresponding to data stored in the memory cell. The threshold voltage of each memory cell included in this storage location is, therefore, a threshold voltage corresponding to any one of the Er state, the A state, the B state, the C state, . . . , and the O state. A storage location in the full sequence programmed state or in the fine programmed state is unavailable for writing new data unless a data erase operation is executed on the block including this storage location.

[0150] Threshold voltage distributions of a plurality of memory cells included in a storage location on which a foggy program operation has been completed will then be described. FIG. 11 illustrates threshold voltage distributions of a plurality of memory cells included in a storage location in a foggy programmed state on which a foggy program operation has been completed, in the memory system 3 according to the first embodiment. A storage location in the foggy programmed state is a storage location where the foggy program operation has been completed but a fine program operation has not been started yet.

[0151] The threshold voltage of each memory cell included in the storage location in the foggy programmed state is set with low precision. The storage location in the foggy programmed state is,

therefore, a storage location where data cannot be read correctly. When data is read from the storage location in the foggy programmed state, the ECC circuit 55 cannot correct an error in the read data. The storage location in the foggy programmed state may include a memory cell having a threshold voltage equal to or higher than the erased page threshold voltage. The controller 5 is thus able to distinguish the storage location in the foggy programmed state from the storage location in the erased state.

[0152] The storage location in the foggy programmed state is a storage location on which the second step program operation (e.g., fine program operation) is executable. In other words, the storage location in the foggy programmed state is not a storage location in the erased state but a storage location on which the fine program operation is executable.

[0153] Threshold voltage distributions of a plurality of memory cells included in a storage location in a state where power loss occurred during a data write operation will then be described. FIG. 12 illustrates threshold voltage distributions of a plurality of memory cells included in a storage location where power loss occurred during a data write operation, in the memory system 3 according to the first embodiment.

[0154] A storage location where a data write operation (a full sequence program operation, a foggy program operation, or a fine program operation) is interrupted because of shutoff of power supply to the memory system 3 during execution of the data write operation is a storage location in a program interrupted state storing data that cannot be read correctly. When data is read from the storage location in the program interrupted state, the ECC circuit 55 cannot correct an error in the read data. Threshold voltages of some memory cells included in the storage location in the program interrupted state exceed the erased page threshold voltage. The controller 5, therefore, can distinguish the storage location in the program interrupted state from the storage location in the erased state.

[0155] Because the storage location in the program interrupted state is not in the erased state, the storage location in the program interrupted state is unavailable for writing new data.

[0156] Hereinafter, a state in which a storage location in the program interrupted state is not present, that is, a case where each storage location is in one of the full sequence programmed state, the fine programmed state, the foggy programmed state, and the erased state will be referred to as a state in which a write operation is stopped properly.

[0157] In both a case where data is read from a storage location in the program interrupted state and a case where data is read from the storage location in the foggy programmed state, the ECC circuit 55 determines an ECC error and the erased page check determines that the storage location is not in the erased state. Only from these two determination results, the controller 5 is not able to correctly determine whether the storage location is in the program interrupted state or in the foggy programmed state.

[0158] A data write operation corresponding to a write command received from the host 2 and a data write operation internally executed according to a garbage collection operation will then be described. FIG. 13 illustrates an example of a data write operation corresponding to a write command received from the host 2 and a data write operation internally executed according to a garbage collection operation, which are executed in the memory system 3 according to the first embodiment. The data write operation corresponding to a write command received from the host 2, that is, the data write operation executed on the basis of the write command received from the host 2 will first be described.

[0159] The controller 5 acquires, from the memory 22 of the host 2, write data associated with the write command received from the host 2, and temporarily stores the acquired write data to a write buffer (WB). A part of the storage area of the DRAM 7 or a part of the storage area of the SRAM 53 may be used as the write buffer.

[0160] The controller 5 selects an SLC block from the free block list 73, and allocates the selected SLC block as a write destination block. The SLC block is a block in which data is to be written in

the SLC mode among blocks included in the NAND memory 6. When an SLC block is already allocated as the write destination block, the controller 5 skips the operation of selecting the SLC block.

[0161] When the total size of write data stored in the write buffer reaches the size of a write unit (e.g., one page), the controller 5 writes the write data into the SLC block, which is allocated as the write destination block. At this time, the controller 5 executes a data write operation in the SLC mode.

[0162] When the SLC block allocated as the write destination block becomes full with data, the controller 5 registers the SLC block in the SLC active block list 74 and deallocates the SLC block from the write destination block. The controller 5 then selects another SLC block from the free block list 73, and allocates the selected another SLC block as a write destination block.

[0163] In a period of idle state or in response to the number of SLC blocks (free SLC blocks) included in the free block list 73 becoming lower than a threshold, the controller 5 executes a garbage collection operation on SLC blocks included in the SLC active block list 74. The idle state means a state in which no command from the host 2 to execute is present.

[0164] The controller 5 selects a QLC block from the free block list 73, and allocates the QLC block as a copy destination block. The QLC block is a block on which foggy-fine program operations in the QLC mode are to be executed among blocks included in the NAND memory 6. Each of memory cells included in the QLC block is to store 4-bit data. Thus, the capacity of the QLC block is four times as much as the SLC block. When a QLC block is already allocated as the copy destination block, the controller 5 skips the operation of selecting the QLC block.

[0165] The controller 5 selects a certain block from the SLC active block list 74, and allocates the selected block as a copy source block. The controller 5 may preferentially select a block that stores a smaller amount of valid data. The controller 5 may select a block to be allocated as the copy source block on the basis of a certain algorithm.

[0166] The controller 5 selects valid data stored in a storage location (copy source storage location) in the copy source block, and reads the selected valid data. The controller 5 writes the read valid data into a storage location (copy destination storage location) into the copy destination block with foggy-fine program operations in the QLC mode. The controller 5 updates the L2P table 71 so that the physical address associated with the logical address corresponding to the valid data is changed from the physical address indicating the copy source storage location to the physical address indicating the copy destination storage location. By this updating, the valid data stored in the copy source storage location changes to invalid data.

[0167] When the copy source block becomes a block storing no valid data, the controller 5 returns the block to the free block list 73 to deallocate the block from the copy source block.

[0168] The controller 5 then selects a certain block from the SLC active block list 74 again, and allocates the selected block as a copy source block.

[0169] When the QLC block allocated as the copy destination block becomes full with data, the controller 5 registers the QLC block in the QLC active block list 75 and deallocates the QLC block from the copy destination block.

[0170] The case where the controller 5 executes the foggy-fine program operations when writing data into the QLC block according to the garbage collection operation has been described above. However, in a case where writing of data based on a write command received from the host 2 is executed, a QLC block may be allocated as the write destination block, as in the case of writing of data according to the garbage collection operation. An operation of writing data into a write destination block will hereinafter be described. The write destination block may be a copy destination block in the garbage collection operation, or may be a block into which data from the host 2 is to be written.

[0171] As described above, in the memory system 3, not only a data write operation in the SLC mode but also a data write operation in the QLC mode is executed. The data write operation in the

QLC mode is executed by using multi-step program operation.

[0172] An operation that is performed in a case where power loss occurred during a data write operation using multi-step program operation will then be described.

[0173] FIG. 14 illustrates a relationship between foggy-fine program operations and an unexpected power loss determination process, which are executed in a memory system according to a comparative example.

[0174] A case is assumed where power supply to the memory system is shut off in a period between completion of a foggy program operation on a storage location (MG (WL N+2, SU2)), which is connected to a word line WL N+2 (N denotes an integer of one or more) and is included in a string unit SU2, and the start of a fine program operation on a storage location (MG (WL N+1, SU)) that is connected to a word line WL N+1 and is included in a string unit SU2.

[0175] In response to resuming of power supply to the memory system after shutoff of power supply to the memory system, a controller acquires a write destination storage location stored in a NAND memory, and determines whether data stored in each of storage locations subsequent to the write destination storage location is readable data.

[0176] The controller thus determines that four storage locations (MG (WL N, SU0) to MG (WL N, SU3)), which are connected to a word line WL N, and two storage locations (MG (WL N+1, SU0) and MG (WL N+1, SU1)) out of four storage locations (MG (WL N+1, SU0) to MG (WL N+1, SU3)), which are connected to the word line WL N+1, are each in a fine programmed state. The storage location that has transitioned to the fine programmed state last is the MG (WL N+1, SU1).

[0177] The controller then executes an erased page check of each storage location, which is subsequent to the MG (WL N+1, SU1) having transitioned to the fine programmed state last, thereby determining whether each storage location subsequent to the MG (WL N+1, SU1) is in an erased state.

[0178] Thus, the controller determines that the MG (WL N+1, SU2), the MG (WL N+1, SU3), the MG (WL N+2, SU0), the MG (WL N+2, SU1), and the MG (WL N+2, SU2) are not in the erased state.

[0179] The controller, however, cannot determine whether each storage location not in the erased state is in a foggy programmed state or in a program interrupted state.

[0180] From the information obtained by the above determination, the controller cannot determine that each of the MGs (WL N+1, SU2) to (WL N+2, SU2) is in any of (1) a state in which power loss occurred during execution of a fine program operation on the MG (WL N+1, SU2), (2) a state in which power loss occurred during execution of a foggy program operation on the MG (WL N+2, SU2) and (3) a state in which power loss occurred during a period between completion of the foggy program operation on the MG (WL N+2, SU2) and the start of the fine program operation on the MG (WL N+1, SU2) but the write operation has been stopped properly.

[0181] In other words, the controller cannot determine whether the previous write operation has been stopped properly.

[0182] To deal with this problem, the memory system 3 according to the first embodiment executes a data write operation according to a rule to execute a foggy program operation and a fine program operation, which becomes executable upon completion of the foggy program operation, as a series of program operations.

[0183] FIG. 15 illustrates a first example of a relationship between the foggy-fine program operations and an unexpected power loss determination process, which are executed in the memory system 3 according to the first embodiment.

[0184] According to the writing order of the foggy-fine program operations described with reference to FIG. 8, the controller 5 executes a data write operation on a plurality of storage locations included in a write destination block. In response to completion of the foggy program operation on the MG (WL N+2, SU2), the controller 5 executes the fine program operation on the MG (WL N+1, SU2). In other words, the controller 5 executes the foggy program operation on the

MG (WL N+2, SU2) and the fine program operation on the MG (WL N+1, SU2), as a series of program operations.

[0185] Now a case is assumed where power supply to the memory system 3 is shut off because of unexpected power loss after completion of the fine program operation on the MG (WL N+1, SU2).

[0186] In response to recovering of power supply to the memory system 3, that is, in response to resuming of power supply to the memory system 3 after power supply to the memory system 3 is shut off, the controller 5 acquires information indicating a write destination storage location from the NAND memory 6. The controller 5 reads data from a storage location indicated by the acquired information, and determines whether the data stored in the storage location is readable data on the basis of a result of error correction (ECC pass or ECC error) by the ECC circuit 55. When the data stored in the storage location is readable data, that is, when the result indicates the ECC pass, the controller 5 reads data stored in a storage location subsequent to this storage location, and determines again whether the data stored in the subsequent storage location is readable data on the basis of a result of an error correction by the ECC circuit 55.

[0187] In this manner, until finding the first storage location that stores unreadable data, the controller 5 repeats to determine whether data stored in a storage location of reading target is readable data while changing the storage location of reading target. Through this process, the controller 5 identifies the last storage location that stores readable data. In other words, the controller 5 identifies the storage location storing data that has become readable last, among a plurality of storage locations included in the write destination block. In this example, the controller 5 identifies the MG (WL N+1, SU2) as the storage location storing data that has become readable last.

[0188] Subsequently, the controller 5 determines whether a first condition that (1-A) among four storage locations that are connected to a word line subsequent to the word line that is connected to the identified storage location, a storage location included in the same string unit that includes the identified storage location stores unreadable data and (1-B) a storage location on which the foggy program operation is to be executed subsequently to execution of the foggy program operation on the above storage location that stores the unreadable data is in the erased state, is satisfied.

[0189] Specifically, the controller 5 first reads data from the MG (WL N+2, SU2), and determines whether the MG (WL N+2, SU2) stores unreadable data on the basis of a result of error correction (ECC pass or ECC error) by the ECC circuit 55.

[0190] When the MG (WL N+2, SU2) stores unreadable data, the controller 5 determines whether the MG (WL N+2, SU3), which is a storage location on which the foggy program operation is to be executed subsequently to execution of the foggy program operation on the MG (WL N+2, SU2), is in the erased state.

[0191] Because the last storage location that stores readable data is the MG (WL N+1, SU2), the foggy program operation on the MG (WL N+2, SU2) is already completed. Thus, when the MG (WL N+2, SU2) stores unreadable data and the MG (WL N+2, SU3) is in the erased state, the controller 5 can determine that the foggy program operation on the MG (WL N+2, SU2) is completed.

[0192] In other words, when the MG (WL N+2, SU2) stores unreadable data and the MG (WL N+2, SU3) is in the erased state, the controller 5 can determine that the MG (WL N+2, SU2) is in the foggy programmed state. Because the MG (WL N+2, SU3) is in the erased state, the fine program operation on the MG (WL N+1, SU3) is not started. The controller 5, therefore, can determine that the MG (WL N+1, SU3) is in the foggy programmed state, too.

[0193] When the MG (WL N+2, SU2) stores unreadable data and the MG (WL N+2, SU3) is in the erased state, the first condition is satisfied. In this case, therefore, the controller 5 determines that the previous write operation in the memory system 3 was stopped properly, that is, a storage location in the program interrupted state is not present in the block allocated as the write destination block. The controller 5 then determines the MG (WL N+2, SU3), which is determined

to be in the erased state, as the next write destination storage location.

[0194] Specifically, when determining that the previous write operation in the memory system 3 has been stopped properly, the controller 5 continues the data write operation on the block allocated as the write destination block, and determines the MG (WL N+2, SU3), as a storage location on which the next foggy program operation is to be executed.

[0195] When the first condition is not satisfied, the controller 5 determines that a storage location where power loss occurred during execution of the foggy program operation or the fine program operation is present, that is, the previous write operation in the memory system 3 was stopped improperly. For example, when the MG (WL N+2, SU2) stores unreadable data but the MG (WL N+2, SU3) is not in the erased state, the controller 5 determines that the previous write operation was stopped improperly.

[0196] The controller 5 may determine whether a condition that the MG (WL N+2, SU2) is not in the erased state and stores unreadable data and that the MG (WL N+2, SU3) is in the erased state, is satisfied. When this condition is satisfied, the controller 5 may determine that the previous write operation in the memory system 3 was stopped properly.

[0197] A case of determining that the previous write operation in the memory system 3 was stopped improperly will then be described. FIG. 16 illustrates a second example of the relationship between the foggy-fine program operations and the unexpected power loss determination process, which are executed in the memory system 3 according to the first embodiment

[0198] It is assumed in this example that power loss occurred during execution of the foggy program operation on the MG (WL N+2, SU2).

[0199] When the memory system 3 restarts after power supply to the memory system 3 is shut off, the controller 5 identifies the last storage location that stores readable data, according to the same procedure as described with reference to FIG. 15. In this case, the controller 5 identifies the MG (WL N+1, SU1) as the last storage location that stores readable data. The controller 5 then determines whether, among four storage locations that are connected to a word line subsequent to the word line that is connected to the identified storage location, a storage location included in the same string unit that includes the identified storage location stores unreadable data. In other words, the controller 5 determines whether the MG (WL N+2, SU1) stores unreadable data.

[0200] When the MG (WL N+2, SU1) stores unreadable data, the controller 5 determines whether the MG (WL N+2, SU2), which is a storage location on which the foggy program operation is to be executed subsequently to execution of the foggy program operation on the MG (WL N+2, SU1), is in the erased state.

[0201] In a case where power loss occurred during execution of the foggy program operation on the MG (WL N+2, SU2), data cannot be read correctly from the MG (WL N+2, SU2). In addition, the MG (WL N+2, SU2) includes one or more memory cells each having a threshold voltage higher than the erased page threshold voltage. It is therefore determined that the MG (WL N+2, SU2) is not in the erased state. On the basis of this result, the controller 5 determines that the foggy program operation on the MG (WL N+2, SU2) and the fine program operation on the MG (WL N+1, SU2) have not been completed as a series of program operations. In this case, the controller 5 is not able to determine whether the MG (WL N+2, SU2) is in the foggy programmed state or power loss occurred during execution of the foggy program operation on the MG (WL N+2, SU2).

[0202] The controller 5 is also not able to determine whether the MG (WL N+1, SU2) is in the foggy programmed state or power loss occurred during execution of the fine program operation on the MG (WL N+1, SU2). This is because data stored in the MG (WL N+1, SU2) is unreadable data, and the MG (WL N+2, SU2) is not in the erased state. Therefore, the MG (WL N+1, SU2) may be in the foggy programmed state or may be in the program interrupted state resulting from power loss that occurred during execution of the fine program operation.

[0203] In this manner, when finding a possibility that a storage location in the program interrupted state may be present in the block allocated as the write destination block, the controller 5

determines that the previous write operation in the memory system 3 was stopped improperly.

[0204] When determining that the previous write operation was stopped improperly, the controller 5 registers the block, which is allocated as the write destination block, in the block management table 72 as an unavailable block. The controller 5 thus deallocates the registered block from the write destination block. The controller 5 then selects another block from the free block list 73. The controller 5 newly allocates the selected block as the write destination block. The controller 5 determines the MG (WL0, SU0), which is the head storage location among storage locations included in the newly allocated block, as the next write destination storage location. Also in a case where data stored in the MG (WL N+2, SU1) is readable data, the first condition is not satisfied. It is determined in this case that the previous write operation was stopped improperly.

[0205] Another case of determining that the previous write operation in the memory system 3 was stopped improperly will then be described. FIG. 17 illustrates a third example of the relationship between the foggy-fine program operations and the unexpected power loss determination process, which are executed in the memory system 3 according to the first embodiment.

[0206] It is assumed in this example that power loss occurred during execution of a fine program operation on the MG (WL N+1, SU2).

[0207] When the memory system 3 restarts after power supply to the memory system 3 is shut off, the controller 5 identifies the last storage location that stores readable data according to the same procedure as described with reference to FIG. 15. In a case where power loss occurred during execution of the fine program operation on the MG (WL N+1, SU2), data cannot be read correctly from the MG (WL N+1, SU2). The controller 5 thus identifies the MG (WL N+1, SU1) as the last storage location that stores readable data.

[0208] The controller 5 then determines whether, among four storage locations that are connected to a word line subsequent to the word line that is connected to the identified storage location, a storage location included in the same string unit that includes the identified storage location stores unreadable data. In other words, the controller 5 determines whether the MG (WL N+2, SU1) stores unreadable data.

[0209] When the MG (WL N+2, SU1) stores unreadable data, the controller 5 determines whether the MG (WL N+2, SU2), which is a storage location on which the foggy program operation is to be executed following execution of the foggy program operation on the MG (WL N+2, SU1), is in the erased state.

[0210] Because the MG (WL N+2, SU2) is in the foggy programmed state, data cannot be read correctly from the MG (WL N+2, SU2). In addition, the MG (WL N+2, SU2) includes one or more memory cells each having a threshold voltage higher than the erased page threshold voltage. It is therefore determined that the MG (WL N+2, SU2) is not in the erased state. On the basis of this result, the controller 5 determines that the foggy program operation on the MG (WL N+2, SU2) and the fine program operation on the MG (WL N+1, SU2) have not been completed as a series of program operations. In this case, the controller 5 is not able to determine whether the MG (WL N+2, SU2) is in the foggy programmed state or power loss occurred during execution of the foggy program operation on the MG (WL N+2, SU2).

[0211] The controller 5 is also not able to determine whether the MG (WL N+1, SU2) is in the foggy programmed state or power loss occurred during execution of the fine program operation on the MG (WL N+1, SU2). This is because data stored in the MG (WL N+1, SU2) is unreadable data, and the MG (WL N+2, SU2) is not in the erased state. Therefore, the MG (WL N+1, SU2) may be in the foggy programmed state or may be in the program interrupted state resulting from power loss that occurred during execution of the fine program operation.

[0212] In this manner, when finding a possibility that a storage location in the program interrupted state may be present in the block allocated as the write destination block, the controller 5 determines that the previous write operation in the memory system 3 was stopped improperly.

[0213] A still another case of determining that the previous write operation in the memory system 3

was stopped improperly will then be described. FIG. 18 illustrates a fourth example of the relationship between the foggy-fine program operations and the unexpected power loss determination process, which are executed in the memory system 3 according to the first embodiment.

[0214] It is assumed in this example that power loss occurred in a period between completion of the foggy program operation on the MG (WL N+2, SU2) and the start of execution of the fine program operation on the MG (WL N+1, SU2).

[0215] When the memory system 3 restarts after power supply to the memory system 3 is shut off, the controller 5 identifies the last storage location that stores readable data according to the same procedure as described with reference to FIG. 15. In this case, the controller 5 identifies the MG (WL N+1, SU1) as the last storage location that stores readable data.

[0216] The controller 5 then determines whether, among four storage locations that are connected to a word line subsequent to the word line that is connected to the identified storage location, a storage location included in the same string unit that includes the identified storage location stores unreadable data. In other words, the controller 5 determines whether the MG (WL N+2, SU1) stores unreadable data.

[0217] When the MG (WL N+2, SU1) stores unreadable data, the controller 5 determines whether the MG (WL N+2, SU2), which is a storage location on which the foggy program operation is to be executed following execution of the foggy program operation on the MG (WL N+2, SU1), is in the erased state.

[0218] Because the MG (WL N+2, SU2) is in the foggy programmed state, data cannot be read correctly from the MG (WL N+2, SU2). In addition, the MG (WL N+2, SU2) includes one or more memory cells each having a threshold voltage higher than the erased page threshold voltage. It is therefore determined that the MG (WL N+2, SU2) is not in the erased state. On the basis of this result, the controller 5 determines that the foggy program operation on the MG (WL N+2, SU2) and the fine program operation on the MG (WL N+1, SU2) have not been completed as a series of program operations. In this case, the controller 5 is not able to determine whether the MG (WL N+2, SU2) is in the foggy programmed state or power loss occurred during execution of the foggy program operation on the MG (WL N+2, SU2).

[0219] The controller 5 is also not able to determine whether the MG (WL N+1, SU2) is in the foggy programmed state or power loss occurred during execution of the fine program operation on the MG (WL N+1, SU2). This is because data stored in the MG (WL N+1, SU2) is unreadable data, and the MG (WL N+2, SU2) is not in the erased state. Therefore, the MG (WL N+1, SU2) may be in the foggy programmed state or may be in the program interrupted state resulting from power loss that occurred during execution of the fine program operation.

[0220] In this manner, when finding a possibility that a storage location in the program interrupted state may be present in the block allocated as the write destination block, the controller 5 determines that the previous write operation in the memory system 3 was stopped improperly.

[0221] A data write operation on four storage locations connected to the head word line of a write destination block will then be described. The data write operation on four storage locations connected to the head word line of a block, which is executed in the memory system according to the comparative example, will first be described. FIG. 19 illustrates a relationship between foggy-fine program operations executed on the head word line of a write destination block and the unexpected power loss determination process, which are executed in the memory system according to the comparative example.

[0222] Now a case is assumed where power loss of the memory system occurred in a period between completion of the foggy program operation on the MG (WL0, SU2) and the start of the foggy program operation on the MG (WL0, SU3).

[0223] Every time data to be written to one storage location among four storage locations, which are connected to the word line WL0, is prepared, the controller of the memory system according to

the comparative example writes the data into the one storage location connected to the word line WL0 by using the foggy program operation. In this case, therefore, power loss of the memory system may occur in a state where the fine program operation on any one of the four storage locations, which are connected to the word line WL0, has not been completed yet.

[0224] In the case of FIG. 19, after the restart of the memory system, the controller cannot determine whether the MG (WL0, SU2) is in the foggy programmed state or in the program interrupted state in which power loss occurred during execution of the foggy program operation.

[0225] To avoid such a situation, according to the memory system 3 of the first embodiment, the foggy program operation on the four storage locations connected to the head word line of a write destination block is started in response to the first fine program operation becoming executable. FIG. 20 illustrates a first example of a relationship between foggy-fine program operations executed on the head word line of the write destination block and the unexpected power loss determination process, which are executed in the memory system 3 according to the first embodiment.

[0226] The controller 5 executes the foggy program operations on four storage locations (MG (WL0, SU0) to MG (WL0, SU3)) connected to the word line WL0, which is the head word line, the foggy program operation on a storage location (MG (WL1, SU0)) connected to the word line WL1 and included in the string unit SU0, and the fine program operation on the MG (WL0, SU0), as a series of program operations. In other words, the controller 5 waits until data to be written to the MG (WL0, SU0) to the MG (WL0, SU3) and data to be written to the MG (WL1, SU0) are prepared. For example, the controller 5 waits for all of these data to be stored in the write buffer. In response to preparation of all the data, the controller 5 executes the foggy program operations on the MG (WL0, SU0) to the MG (WL0, SU3), the foggy program operation on the MG (WL1, SU0), and the fine program operation on the MG (WL0, SU0).

[0227] The controller 5 is thus able to complete the fine program operation on the MG (WL0, SU0) unless unexpected power loss occurs. When data stored in the MG (WL0, SU0) is readable data, the controller 5 can execute the unexpected power loss determination process on the write destination block, according to the same procedure as described with reference to FIG. 15. At the restart of the memory system 3, when finding that data stored in the MG (WL0, SU0) is unreadable data and that the MG (WL0, SU0) is not in the erased state, the controller 5 can determine that the previous write operation in the memory system 3 was stopped improperly.

[0228] Hereinafter, this case will be described in detail. It is assumed in the example of FIG. 20 that following completion of the fine program operation on the MG (WL0, SU0), power supply to the memory system 3 is shut off because of unexpected power loss.

[0229] When the memory system 3 restarts after power supply to the memory system 3 is shut off, the controller 5 identifies the last storage location that stores readable data according to the same procedure as described with reference to FIG. 15. In this case, the controller 5 identifies the MG (WL0, SU0) as the last storage location that stores readable data.

[0230] The controller 5 then determines whether, among four storage locations connected to a word line subsequent to the word line connected to the identified storage location, a storage location included in the same string unit that includes the identified storage location stores unreadable data. In other words, in this case, the controller 5 determines whether the MG (WL1, SU0) stores unreadable data.

[0231] When the MG (WL1, SU0) stores unreadable data, the controller 5 determines whether the MG (WL1, SU1), which is a storage location on which the foggy program operation is to be executed following execution of the foggy program operation on the MG (WL1, SU0), is in the erased state

[0232] Because the last storage location that stores readable data is the MG (WL0, SU0), the foggy program operation on the MG (WL1, SU0) is already completed. Thus, when the MG (WL1, SU0) stores unreadable data and the MG (WL1, SU1) is in the erased state, the controller 5 can

determine that the foggy program 1C operation on the MG (WL1, SU0) is completed.

[0233] In other words, when the MG (WL1, SU0) stores unreadable data and the MG (WL1, SU1; is in the erased state, the controller 5 can determine that the MG (WL1, SU0) is in the foggy programmed state. Because the MG (WL1, SU1) is in the erased state, the fine program operation on the MG (WTO, SU1) is not started. The controller 5, therefore, can determine that the MG (WL0, SU1) is in the foggy programmed state, too.

[0234] When the MG (WL1, SU0) stores unreadable data and the MG (WL1, SU1) is in the erased state, the first condition is satisfied. In this case, therefore, the controller 5 determines that the previous write operation in the memory system 3 was stopped properly, that is, a storage location in the program interrupted state is not present in the block allocated as the write destination block. The controller 5 then determines the MG (WL1, SU1), which is determined to be in the erased state, as the next write destination storage location.

[0235] Specifically, when determining that the previous write operation in the memory system 3 was stopped properly, the controller 5 continues the data write operation on the block allocated as the write destination block, and determines the MG (WL1, SU1) as the storage location on which the next foggy program operation is to be executed.

[0236] When the first condition is not satisfied, the controller 5 determines that a storage location where power loss occurred during execution of the foggy program operation or the fine program operation is present, that is, the previous write operation in the memory system 3 was stopped improperly. For example, when the MG (WL1, SU0) stores unreadable data but the MG (WL1, SU1) is not in the erased state, the controller 5 determines that the previous write operation was stopped improperly.

[0237] The controller 5 may determine whether a condition that the MG (WL1, SU0) is not in the erased state and stores unreadable data and that the MG (WL1, SU1) is in the erased state, is satisfied. When this condition is satisfied, the controller 5 may determine that the previous write operation in the memory system 3 was stopped properly.

[0238] A case of determining that the previous write operation in the memory system 3 was stopped improperly will then be described. FIG. 21 illustrates a second example of the relationship between the foggy-fine program operations on the head word line of the write destination block and the unexpected power loss determination process, which are executed in the memory system 3 according to the first embodiment.

[0239] It is assumed in this example that power loss occurred during execution of the foggy program operation on the MG (WL1, SU0).

[0240] When the memory system 3 restarts after power supply to the memory system 3 is shut off, the controller 5 identifies the last storage location that stores readable data according to the same procedure as described with reference to FIG. 15. In the case of FIG. 21, none of the storage locations stores readable data. The controller 5, therefore, determines that a storage location that stores readable data, i.e., a storage location in the fine programmed state is not present. The controller 5 thus determines that the foggy program operations on the MG (WL0, SU0) to MG (WL0, SU3), the foggy program operation on the MG (WL1, SU0), and the fine program operation on the MG (WL0, SU0) have not been completed as a series of program operations. In this case, the controller 5 is not able to determine whether the MG (WL1, SU0) is in the foggy programmed state or power loss occurred during execution of the foggy program operation on the MG (WL1, SU0).

[0241] The controller 5 is also not able to determine whether the MG (WL0, SU0) is in the foggy programmed state or power loss occurred during execution of the fine program operation on the MG (WL0, SU0).

[0242] In this manner, when finding a possibility that a storage location in the program interrupted state may be present in the block allocated as the write destination block, the controller 5 determines that the previous write operation in the memory system 3 was stopped improperly.

[0243] When determining that the previous write operation was stopped improperly, the controller 5 registers the block, which is allocated as the write destination block, in the block management table 72 as an unavailable block. The controller 5 thus deallocates the registered block from the write destination block. The controller 5 then selects another block from the free block list 73. The controller 5 newly allocates the selected block as the write destination block. The controller 5 determines the MG (WL0, SU0), which is the head storage location among storage locations included in the newly allocated block, as the next write destination storage location.

[0244] Another case of determining that the previous write operation in the memory system 3 was stopped improperly will then be described. FIG. 22 illustrates a third example of the relationship between the foggy-fine program operations on the head word line of the write destination block and the unexpected power loss determination process, which are executed in the memory system 3 according to the first embodiment.

[0245] It is assumed in this example that power loss occurred during execution of the fine program operation on the MG (WL0, SU0).

[0246] When the memory system 3 restarts after power supply to the memory system 3 is shut off, the controller 5 identifies the last storage location that stores readable data according to the same procedure as described with reference to FIG. 15. In the case of FIG. 22, none of the storage locations stores readable data. The controller 5, therefore, determines that a storage location that stores readable data, i.e., a storage location in the fine programmed state is not present. The controller 5 thus determines that the foggy program operations on the MG (WL0, SU0) to MG (WL0, SU3), the foggy program operation on the MG (WL1, SU0), and the fine program operation on the MG (WL0, SU0) have not been completed as a series of program operations.

[0247] In this case, the controller 5 is not able to determine whether the MG (WL0, SU0) is in the foggy programmed state or power loss occurred during execution of the foggy program operation on the MG (WL0, SU0). The controller 5 is also not able to determine whether the MG (WL1, SU0) is in the foggy programmed state or power loss occurred during execution of the fine program operation on the MG (WL1, SU0).

[0248] In this manner, when finding a possibility that a storage location in the program interrupted state may be present in the block allocated as the write destination block, the controller 5 determines that the previous write operation in the memory system 3 was stopped improperly.

[0249] A still another case of determining that the previous write operation in the memory system 3 was stopped improperly will then be described. FIG. 23 illustrates a fourth example of the relationship between the foggy-fine program operations on the head word line of the write destination block and the unexpected power loss determination process, which are executed in the memory system 3 according to the first embodiment.

[0250] It is assumed in this example that power loss occurred in a period between completion of the foggy program operation on the MG (WL1, SU0) and the start of the fine program operation on the MG (WL0, SU0).

[0251] When the memory system 3 restarts after power supply to the memory system 3 is shut off, the controller 5 identifies the last storage location that stores readable data according to the same procedure as described with reference to FIG. 15. In the case of FIG. 23, none of the storage locations stores readable data. The controller 5, therefore, determines that a storage location that stores readable data, i.e., a storage location in the fine programmed state is not present. The controller 5 thus determines that the foggy program operations on the MG (WL0, SU0) to MG (WL0, SU3), the foggy program operation on the MG (WL1, SU0), and the fine program operation on the MG (WL0, SU0) have not been completed as a series of program operations.

[0252] In this case, the controller 5 is not able to determine whether the MG (WL1, SU0) is in the foggy programmed state or power loss occurred during execution of the foggy program operation on the MG (WL1, SU0). The controller 5 is also not able to determine whether the MG (WL0, SU0) is in the foggy programmed state or power loss occurred during execution of the fine program

operation on the MG (WL0, SU0).

[0253] In this manner, when there is a possibility that a storage location in the program interrupted state may be present in the block allocated as the write destination block, the controller 5 determines that the previous write operation in the memory system 3 was stopped improperly.

[0254] The cases where the fine program operation on the MG (WL0, SU0) becomes executable after completion of the foggy program operation on the MG (WL1, SU0) have been described with reference to FIGS. 20 to 23. The foggy-fine program operations, however, may be executed in an order different from the order indicated in FIGS. 20 to 23. For example, the fine program operation on the MG (WL0, SU0) may become executable after completion of the foggy program operation on the MG (WL0, SU1). In this case, the foggy program operation on the MG (WL0, SU0) is started when data to be written to the MG (WL0, SU0) and MG (WL0, SU1) have been prepared.

[0255] A case where the number of string units included in each block is one will then be described. FIG. 24 illustrates a first example of a relationship between foggy-fine program operations and the unexpected power loss determination process, which are executed in the memory system 3 according to the first embodiment, in a case where each block includes only one string unit SU. Hereinafter, a storage location connected to a certain word line WLx will be referred to as MG (WLx).

[0256] When each block includes only one string unit SU, the controller 5 executes a data write operation by executing the foggy program operation on an MG (WL0), the foggy program operation on an MG (WL1), the fine program operation on an MG (WL0), the foggy program operation on an MG (WL2), and the fine program operation on an MG (WL1) in this order.

[0257] The controller 5 executes the foggy program operation and the fine program operation which becomes executable upon completion of the foggy program operation, as a series of program operations. The controller 5 thus executes the foggy program operation on the MG (WL1) and the fine program operation on the MG (WL0), as a series of program operations.

[0258] Hereinafter, this case will be described in detail. It is assumed in FIG. 24 that following completion of the fine program operation on an MG (WL6), power supply to the memory system 3 has been shut off because of unexpected power loss.

[0259] When the memory system 3 restarts after power supply to the memory system 3 is shut off, the controller 5 identifies the last storage location that stores readable data according to the same procedure as described with reference to FIG. 15. In this case, the controller 5 identifies the MG (WL6) as the last storage location that stores readable data.

[0260] The controller 5 then determines whether, among storage locations that are connected to a word line subsequent to the word line that is connected to the identified storage location, a storage location included in the same string unit that includes the identified storage location stores unreadable data. Because only one string unit is present in this case, the controller 5 just checks a storage location connected to a word line subsequent to the word line connected to the identified storage location. In other words, the controller 5 determines whether an MG (WL7) stores unreadable data.

[0261] When the MG (WL7) stores unreadable data, the controller 5 determines whether an MG (WL8), which is a storage location on which the foggy program operation is to be executed subsequently to execution of the foggy program operation on the MG (WL7), is in the erased state.

[0262] Because the last storage location that stores readable data is the MG (WL6), the foggy program operation on the MG (WL7) is already completed. Thus, when the MG (WL7) stores unreadable data and the MG (WL8) is in the erased state, the controller 5 can determine that the foggy program operation on the MG (WL7) is completed.

[0263] When the MG (WL7) stores unreadable data and the MG (WL8) is in the erased state, the first condition is satisfied. In this case, therefore, the controller 5 determines that the previous write operation in the memory system 3 was stopped properly, that is, a storage location in the program interrupted state is not present in the block allocated as the write destination block. The controller 5

then determines the MG (WL8), which is determined to be in the erased state, as the next write destination storage location.

[0264] When the first condition is not satisfied, the controller 5 determines that a storage location where power loss occurred during execution of the foggy program operation or the fine program operation is present, that is, the previous write operation in the memory system 3 was stopped improperly. For example, when the MG (WL7) stores unreadable data but the MG (WL8) is not in the erased state, the controller 5 determines that the previous write operation was stopped improperly.

[0265] A case of determining that the previous write operation in the memory system 3 was stopped improperly will then be described. FIG. 25 illustrates a second example of the relationship between the foggy-fine program operations and the unexpected power loss determination process, which are executed in the memory system 3 according to the first embodiment, in the case where each block includes only one string unit.

[0266] It is assumed in this example that power loss occurred during execution of the foggy program operation on the MG (WL7).

[0267] When the memory system 3 restarts after power supply to the memory system 3 is shut off, the controller 5 identifies the last storage location that stores readable data according to the same procedure as described with reference to FIG. 15. In this case, the controller 5 identifies the MG (WL5) as the last storage location that stores readable data.

[0268] The controller 5 then determines whether a storage location connected to a word line subsequent to the word line connected to the identified storage location stores unreadable data. In other words, the controller 5 determines whether the MG (WL6) stores unreadable data.

[0269] When the MG (WL6) stores unreadable data, the controller 5 determines whether an MG (WL7), which is a storage location on which the foggy program operation is to be executed subsequently to execution of the foggy program operation on the MG (WL6), is in the erased state.

[0270] In a case where power loss occurred during execution of the foggy program operation on the MG (WL7), the controller 5 is not able to read data correctly from the MG (WL7). In addition, the MG (WL7) includes one or more memory cells each having a threshold voltage higher than the erased page threshold voltage. It is therefore determined that the MG (WL7) is not in the erased state. On the basis of this result, the controller 5 determines that the foggy program operation on the MG (WL7) and the fine program operation on the MG (WL6) have not been completed as a series of program operations. In this case, the controller 5 is not able to determine whether the MG (WL7) is in the foggy programmed state or power loss occurred during execution of the foggy program operation on the MG (WL7).

[0271] The controller 5 is also not able to determine whether the MG (WL6) is in the foggy programmed state or power loss occurred during execution of the fine program operation on the MG (WL6).

[0272] In this manner, when finding a possibility that a storage location in the program interrupted state may be present in the block allocated as the write destination block, the controller 5 determines that the previous write operation in the memory system 3 was stopped improperly.

[0273] When determining that the previous write operation was stopped improperly, the controller 5 registers the block, which is allocated as the write destination block, in the block management table 72 as an unavailable block. The controller 5 thus deallocates the registered block from the write destination block. The controller 5 then selects another block from the free block list 73. The controller 5 newly allocates the selected block as the write destination block. The controller 5 determines the MG (WL0), which is the head storage location among storage locations included in the newly allocated block, as the next write destination storage location.

[0274] Another case of determining that the previous write operation in the memory system 3 was stopped improperly will then be described. FIG. 26 illustrates a third example of the relationship between the foggy-fine program operations and the unexpected power loss determination process,

which are executed in the memory system 3 according to the first embodiment, in the case where each block includes only one string unit.

[0275] It is assumed in this example that power loss occurred during execution of the fine program operation on the MG (WL6).

[0276] When the memory system 3 restarts after power supply to the memory system 3 is shut off, the controller 5 identifies the last storage location that stores readable data according to the same procedure as described with reference to FIG. 15. In a case where power loss occurred during execution of the fine program operation on the MG (WL6), data cannot be read correctly from the MG (WL6). The controller 5 thus identifies the MG (WL5) as the last storage location that stores readable data.

[0277] The controller 5 then determines whether a storage location connected to a word line subsequent to the word line connected to the identified storage location stores unreadable data. In other words, the controller 5 determines whether the MG (WL6) stores unreadable data.

[0278] When the MG (WL6) stores unreadable data, the controller 5 determines whether the MG (WL7), which is a storage location on which the foggy program operation is to be executed following execution of the foggy program operation on the MG (WL6), is in the erased state

[0279] The MG (WL7) is in the foggy programmed state, and therefore data cannot be read correctly from the MG (WL7). In addition, the MG (WL7) includes one or more memory cells each having a threshold voltage higher than the erased page threshold voltage. It is therefore determined that the MG (WL7) is not in the erased state. On the basis of this result, the controller 5 determines that the foggy program operation on the MG (WL7) and the fine program operation on the MG (WL6) have not been completed as a series of program operations. In this case, the controller 5 is not able to determine whether the MG (WL7) is in the foggy programmed state or power loss occurred during execution of the foggy program operation on the MG (WL7).

[0280] The controller 5 is also not able to determine whether the MG (WL6) is in the foggy programmed state or power loss occurred during execution of the fine program operation on the MG (WL6). This is because data stored in the MG (WL6) is unreadable data and the MG (WL7) is not in the erased state. Therefore, the MG (WL6) may be in the foggy programmed state or may be in the program interrupted state in which power loss occurred during execution of the fine program operation.

[0281] In this manner, when there is a possibility that a storage location in the program interrupted state may be present in the block allocated as the write destination block, the controller 5 determines that the previous write operation in the memory system 3 was stopped improperly.

[0282] A still another case of determining that the previous write operation in the memory system 3 was stopped improperly will then be described. FIG. 27 illustrates a fourth example of the relationship between the foggy-fine program operations and the unexpected power loss determination process, which are executed in the memory system 3 according to the first embodiment, in the case where each block includes only one string unit.

[0283] It is assumed in this example that power loss occurred in a period between completion of the foggy program operation on the MG (WL7) and the start of the fine program operation on the MG (WL6).

[0284] When the memory system 3 restarts after power supply to the memory system 3 is shut off, the controller 5 identifies the last storage location that stores readable data according to the same procedure as described with reference to FIG. 15. In this case, the controller 5 identifies the MG (WL5) as the last storage location that stores readable data.

[0285] The controller 5 then determines whether a storage location connected to a word line subsequent to the word line connected to the identified storage location stores unreadable data. In other words, the controller 5 determines whether the MG (WL6) stores unreadable data.

[0286] When the MG (WL6) stores unreadable data, the controller 5 determines whether the MG (WL7), which is a storage location on which the foggy program operation is to be executed

subsequently to execution of the foggy program operation on the MG (WL6), is in the erased state. [0287] The MG (WL7) is in the foggy programmed state, and therefore data cannot be read correctly from the MG (WL7). In addition, the MG (WL7) includes one or more memory cells each having a threshold voltage higher than the erased page threshold voltage. It is therefore determined that the MG (WL7) is not in the erased state. On the basis of this result, the controller 5 determines that the foggy program operation on the MG (WL7) and the fine program operation on the MG (WL6) have not been completed as a series of program operations. In this case, the controller 5 is not able to determine whether the MG (WL7) is in the foggy programmed state or power loss occurred during execution of the foggy program operation on the MG (WL7).

[0288] The controller 5 is also not able to determine whether the MG (WL6) is in the foggy programmed state or power loss occurred during execution of the fine program operation on the MG (WL6). This is because data stored in the MG (WL6) is unreadable data and the MG (WL7) is not in the erased state. Therefore, the MG (WL6) may be in the foggy programmed state or may be in the program interrupted state in which power loss occurred during execution of the fine program operation.

[0289] In this manner, when there is a possibility that a storage location in the program interrupted state may be present in the block allocated as the write destination block, the controller 5 determines that the previous write operation in the memory system 3 was stopped improperly.

[0290] The procedure of the unexpected power loss determination process will then be described. FIG. 28 is a flowchart illustrating the procedure of the unexpected power loss determination process that is executed in the memory system 3 according to the first embodiment. When the memory system 3 restarts, that is, power supply to the memory system 3 is resumed after power supply to the memory system 3 is shut off, the controller 5 executes the unexpected power loss determination process. The unexpected power loss determination process is a process of determining whether the previous write operation in the memory system 3 was stopped properly and correctly determining the next write destination storage location.

[0291] When power supply to the memory system 3 is resumed, the controller 5 acquires information indicating a write destination storage location (step S102). The controller 5 reads the information indicating the write destination storage location, from the NAND memory 6. The information indicating the write destination storage location is periodically stored in the NAND memory 6. A storage location indicated as the write destination storage location by the information acquired from the NAND memory 6 is, therefore, a storage location to which a data write operation had been completed when the previous power loss occurred, among storage locations included in the write destination block.

[0292] The controller 5 starts a data read operation from the storage location indicated by the information acquired in step S102 (step S103). The controller 5 executes the data read operation on the storage location indicated by the information acquired in step S102, and checks a result of error correction (ECC pass or ECC error) by the ECC circuit 55. The controller 5 thus determines whether data stored in the storage location is readable data. When the data stored in the storage location is readable data, the controller 5 executes a data read operation on the next storage location subsequent to this storage location, and checks a result of error correction by the ECC circuit 55. The controller 5 thus determines whether data stored in the next storage location is readable data.

[0293] By repeating the data read operation and check of the error correction result in step S103, the controller 5 identifies the last storage location that stores readable data (step S104). The last storage location that stores readable data is a storage location storing data that becomes readable last, among the storage locations included in the write destination block.

[0294] The controller 5 determines whether data stored in a storage location that is connected to the next word line subsequent to the word line connected to the identified last storage location (i.e., the last storage location that stores readable data) and that is included in the same string unit including the identified last storage location, is readable data (step S105).

[0295] When the data stored in the storage location that is connected to the next word line and included in the same string unit is not readable data (No in step **S105**), the process by the controller **5** proceeds to step **S106**. In this case, the data stored in the storage location that is connected to the next word line and included in the same string unit is unreadable data.

[0296] The controller **5** determines whether a storage location on which the foggy program operation is to be executed subsequently to execution of the foggy program operation on the storage location that stores the unreadable data is in the erased state (step **S106**). The controller **5** executes the erased page check on the storage location on which the foggy program operation is to be executed subsequently to the storage location that stores the unreadable data. By executing the erased page check, the controller **5** determines whether the target storage location includes a memory cell whose threshold voltage is equal to or higher than the erased page threshold voltage. When the storage location does not include a memory cell whose threshold voltage is equal to or higher than the erased page threshold voltage, the controller **5** determines that the storage location is in the erased state.

[0297] When the storage location on which the foggy program operation is to be executed subsequently to the storage location that stores the unreadable data is in the erased state (Yes in step **S106**), that is, when the first condition is satisfied (No in step **S105** and Yes in step **S106**), the controller **5** determines that the previous write operation in the memory system **3** was stopped properly (step **S107**). The controller **5** understands that the previous power loss occurred in a period between completion of the fine program operation on the storage location identified in step **S104** and the start of the foggy program operation on the storage location on which the erased page check in step **S106** is performed. On the basis of this understanding, the controller **5** determines that the previous write operation in the memory system **3** was stopped properly.

[0298] The controller **5** determines the storage location determined to be in the erased state in step **S106**, as the next write destination storage location on which the next foggy program operation is to be executed (step **S108**). In other words, when determining that the previous write operation in the memory system **3** was stopped properly, the controller **5** decides to continue data writing to the block allocated as the write destination block.

[0299] When the data stored in the storage location that is connected to the next word line and included in the same string unit is readable data (Yes in step **S105**), the process by the controller **5** returns to step **S104**.

[0300] In this case, the first condition is not satisfied. Besides, it is determined that the last storage location identified in step **S104** is incorrect one. The controller **5**, therefore, identifies again the last storage location that stores readable data (step **S104**).

[0301] When the storage location on which the foggy program operation is to be executed subsequently to the storage location that is determined to be storing the unreadable data in step **S105** is not in the erased state (No in step **S106**), the controller **5** determines that the previous write operation in the memory system **3** was stopped improperly (step **S109**). The controller **5** understands that the previous power loss to the memory system **3** occurred during a period between the start of the foggy program operation on the storage location determined not to be in the erased state in step **S106** and completion of the fine program operation on the storage location on which the fine program operation is to be executed subsequently to the storage location identified in step **S104**. The controller **5** thus determines that the previous write operation in the memory system **3** was stopped improperly. In this case, there is a possibility that a storage location in the program interrupted state may be present in the block allocated as the write destination block.

[0302] In this case, the first condition is not satisfied. The controller **5**, therefore, determines the block allocated as the write destination block, as an unavailable block (step **S110**). The controller **5** registers this block in the block management table **72** as an unavailable block.

[0303] The controller **5** allocates another block as a new write destination block (step **S111**). The controller **5** selects the block from the free block list **73**. The controller **5** then allocates the selected

block as the write destination block.

[0304] The controller 5 determines the head storage location among a plurality of storage locations included in the block allocated as the write destination block, that is, one of one or more storage locations connected to the head word line, as the next write destination storage location (step S112).

[0305] Basically, the controller 5 may execute the unexpected power loss determination process every time power supply is resumed, regardless of whether the previous power loss is unexpected power loss in which power supply is suddenly shut off or power supply is shut off after receiving a shutdown notice from the host 2.

[0306] Note that the controller 5 may execute the unexpected power loss determination process in response to the restart of power supply only when the previous power loss was the unexpected power loss. When receiving the shutdown notice from the host 2, the controller 5 performs a shutdown preparation process that includes a process of storing necessary data to the NAND memory 6. Therefore, in response to the restart of power supply, the controller 5 can determine, for example, whether information indicating completion of the shutdown preparation process is stored in the NAND memory 6. When the information indicating completion of the shutdown preparation process is not stored in the NAND memory 6, the controller 5 determines that the previous power loss is the unexpected power loss.

[0307] The procedure of the data write operation on the write destination block will then be described. FIG. 29 is a flowchart illustrating the procedure of the data write operation executed in the memory system 3 according to the first embodiment.

[0308] The controller 5 determines whether data to be written into a write destination block has been prepared (step S21). For example, the controller 5 determines whether the size of data to be written has reached the size of a write unit. The size of the write unit is the size of data to be written into one storage location (memory cell group MG) of the write destination block. When data is written to the write destination block in the QLC mode, the size of the write unit is equivalent to the size of four pages.

[0309] When data to be written into the write destination block has not been prepared (No in step S21), the controller 5 waits until the data is prepared.

[0310] When data to be written into the write destination block has been prepared (Yes in step S21), the controller 5 determines whether a storage location of the next writing destination is a storage location connected to the head word line of the write destination block (step S22).

[0311] When the storage location of the next write destination is not connected to the head word line (No in step S22), the controller 5 executes the foggy program operation on the storage location of the next write destination (step S23).

[0312] The controller 5 then executes the fine program operation on a storage location to which the fine program operation has become executable upon having executed the foggy program operation in step S23 (step S24).

[0313] When the storage location of the next write destination is connected to the head word line (Yes in step S22), the controller 5 determines whether the total size of data to be written into the write destination block has reached the size that enables execution of the fine program operation (step S25). For example, in a case where the write destination block includes four string units SU, the data size that enables execution of the fine program operation is equivalent to the size of five storage locations (five memory cell groups MG). When data is written into the write destination block in the QLC mode, therefore, the data size that enables execution of the fine program operation is equivalent to the size of 20 pages.

[0314] When the total size of data has not reached the size that enables execution of the fine program operation (No in step S25), the controller 5 waits until the total size of data reaches the size that enables execution of the fine program operation. In this case, the process by the controller 5 returns to step S25.

[0315] When the total size of data has reached the size that enables execution of the fine program operation (Yes in step S25), the controller 5 executes the foggy program operations until the fine program operation becomes executable (step S23). Specifically, the controller 5 executes the foggy program operations on one or more storage locations that are connected to the head word line of the write destination block and the foggy program operation on one storage location that is connected to the word line subsequent to the head word line.

[0316] The controller 5 then executes the fine program operation on a storage location to which the fine program operation has become executable upon having executed the foggy program operation in step S23 (step S24).

[0317] As described above, according to the first embodiment, the controller 5 can determine whether the previous write operation was stopped properly by using the rule that the foggy program operation and the fine program operation, which becomes executable upon completion of the foggy program operation, are executed as a series of program operations, thereby determining the next write destination storage location correctly.

[0318] A method of determining, in the first embodiment, whether a storage location on which the fine program operation of the series of program operations is to be executed is in the foggy programmed state or in the program interrupted state in which power loss occurred during execution of the fine program operation, will then be described.

[0319] For example, in the case of FIG. 18, the MG (WL N+1, SU2) is a storage location that cannot be determined to be whether in the foggy programmed state or in the program interrupted state in which power loss occurred during execution of the fine program operation.

[0320] When determining that the foggy program operation on the MG (WL N+2, SU2) and the fine program operation on the MG (WL N+1, SU2) have not been completed as a series of program operations, the controller 5 may execute the following operations.

[0321] The controller 5 executes the fine program operation on the MG (WL N+1, SU2). When completing the fine program operation on the MG (WL N+1, SU2), the controller 5 determines whether data is read correctly from the MG (WL N+1, SU2). When data is read correctly from the MG (WL N+1, SU2), the controller 5 can determine that the MG (WL N+1, SU2) was in the foggy programmed state when the previous write operation was stopped.

[0322] A measure by which, in the first embodiment, a series of program operations can be completed even if unexpected power loss occurs during execution of a write operation will then be described.

[0323] As illustrated with a broken line in FIG. 1, the memory system 3 may further include a capacitor 81. The capacitor 81 is an element capable of storing power. The capacitor 81 is electrically connected to the power supply circuit 8. When a voltage supplied from the host 2 drops due to unexpected power loss, the power supply circuit 8 supplies power accumulated in the capacitor 81, to the respective components of the memory system 3.

[0324] When the unexpected power loss has occurred, the controller 5 continues a write operation by using power from the capacitor 81, at least until a series of program operations in progress is completed. For example, in the case of FIG. 16, unexpected power loss occurs during execution of the foggy program operation on the MG (WL N+2, SU2). In this case, the controller 5 continues the foggy program operation on the MG (WL N+2, SU2) by using power from the capacitor 81. When completing the foggy program operation on the MG (WL N+2, SU2), the controller 5 executes the fine program operation on the MG (WL N+1, SU2) by using power from the capacitor 81. Through this process, the series of program operations is completed.

[0325] In the case of FIG. 17, for example, unexpected power loss occurs during execution of the fine program operation on the MG (WL N+1, SU2). In this case, the controller 5 continues the fine program operation on the MG (WL N+1, SU2) by using power from the capacitor 81. Through this process, the series of program operations is completed.

[0326] In the case of FIG. 18, for example, unexpected power loss occurs in a period between

completion of the foggy program operation on the MG (WL N+2, SU2) and the start of the fine program operation on the MG (WL N+1, SU2). In this case, the controller 5 executes the fine program operation on the MG (WL N+1, SU2) by using power from the capacitor 81. Through this process, the series of program operations is completed.

[0327] An operation of writing information which indicates a storage location where a write operation is interrupted, into the NAND memory 6 by using power from the capacitor 81, which is executed in the first embodiment, will then be described.

[0328] For example, in the case of FIG. 16, unexpected power loss occurs during execution of the foggy program operation on the MG (WL N+2, SU2). In this case, the controller 5 writes information which indicates the MG (WL N+2, SU2) where writing of data is interrupted, into the NAND memory 6 by using power from the capacitor 81.

[0329] In the case of FIG. 17, for example, unexpected power loss occurs during execution of the fine program operation on the MG (WL N+1, SU2). In this case, the controller 5 writes information which indicates the MG (WL N+1, SU2) where writing of data is interrupted, into the NAND memory 6 by using power from the capacitor 81.

[0330] In the case of FIG. 18, for example, unexpected power loss occurs in a period between completion of the foggy program operation on the MG (WL N+2, SU2) and the start of the fine program operation on the MG (WL N+1, SU2). In this case, the controller 5 writes information which indicates that the write operation is interrupted between the foggy program operation on the MG (WL N+2, SU2) and the fine program operation on the MG (WL N+1, SU2), into the NAND memory 6.

[0331] In this manner, by writing information on a storage location where the write operation is interrupted, into the NAND memory 6, whether the previous write operation was stopped properly can be easily determined.

Second Embodiment

[0332] A second embodiment will be described. FIG. 30 is a block diagram illustrating an example of a configuration of an information processing system 1 including a memory system 3 according to the second embodiment.

[0333] Most of the components included in the memory system 3 according to the second embodiment are the same as those included in the memory system 3 according to the first embodiment illustrated in FIG. 1. The following description, therefore, will be focused on differences between the memory system 3 according to the second embodiment and the memory system 3 according to the first embodiment.

[0334] In the memory system 3 according to the second embodiment, the controller 5 additionally manages foggy program management information 76. The foggy program management information 76 is stored in, for example, the DRAM 7.

[0335] The foggy program management information 76 is information indicating a storage location in the foggy programmed state among a plurality of storage locations included in a block allocated as a write destination block. Specifically, the foggy program management information 76 indicates each storage location where the foggy program operation is completed and the fine program operation is not started yet.

[0336] An operation of updating the foggy program management information 76 in response to completion of the foggy program operation will then be described. FIG. 31 illustrates the foggy program operation executed in the memory system 3 according to the second embodiment.

[0337] In an initial state illustrated in FIG. 31, the MG (WL N, SU0) connected to the word line WL N is in the fine programmed state. The other three MGs (WL N, SU1), (WL N, SU2), and (WL N, SU3) connected to the word line WL N are each in the foggy programmed state. The MG (WL N+1, SU0) connected to the word line WL N+1 is also in the foggy programmed state. In the initial state illustrated in FIG. 31, therefore, information indicating the MG (WL N, SU1) to the MG (WL N, SU3) and information indicating the MG (WL N+1, SU0) are managed as the foggy program

management information **76**.

[0338] Step 1: The controller **5** executes, for example, the foggy program operation on the MG (WL N+1, SU1).

[0339] Step 2: In response to completion of the foggy program operation, the controller **5** updates the foggy program management information **76** stored in the DRAM **7**. The controller **5** adds information indicating a storage location where the foggy program operation is completed, i.e., information indicating the MG (WL N+1, SU1), to the foggy program management information **76**.

[0340] Step 3: The controller **5** writes the updated foggy program management information **76** into the NAND memory **6**.

[0341] In this manner, the controller **5** updates the foggy program management information **76** in response to completion of the foggy program operation. The controller **5** then stores the updated latest foggy program management information **76** to the NAND memory **6**.

[0342] In this manner, after completion of the foggy program operation, the foggy program management information **76** is updated and is written into the NAND memory **6**. According to this process, for example, if power loss occurred during execution of the foggy program operation on the MG (WL N+1, SU1), information indicating the MG (WL N+1, SU1) is not added to the foggy program management information **76**. When power supply to the memory system **3** is resumed, therefore, the controller **5** reads the foggy program management information **76** from the NAND memory **6**, thereby properly determining each storage location in the foggy programmed state, which is included in a block allocated as a write destination block at the time of power loss. Then, the controller **5** can determine that the MG (WL N+1, SU1) storing unreadable data is in the program interrupted state. Hence, the controller **5** can determine that the previous write operation was stopped improperly.

[0343] An operation of updating the foggy program management information **76** before starting the fine program operation will then be described. FIG. 32 illustrates the fine program operation executed in the memory system **3** according to the second embodiment. It is assumed in FIG. 32 that after the state illustrated in FIG. 31, the fine program operation on the MG (WL N, SU1) is started.

[0344] Step 1: Before starting the fine program operation on the MG (WL N, SU1), the controller **5** updates the foggy program management information **76**. Specifically, the controller **5** deletes information indicating the MG (WL N, SU1) on which the fine program operation is to be executed next, from the foggy program management information **76**.

[0345] Step 2: The controller **5** writes the updated foggy program management information **76** into the NAND memory **6**.

[0346] Step 3: The controller **5** executes the fine program operation on the MG (WL N, SU1).

[0347] In this manner, before starting the fine program operation on a certain storage location, the controller **5** updates the foggy program management information **76** so that information indicating the certain storage location is deleted from the foggy program management information **76**. The controller **5** then stores the updated latest foggy program management information **76** into the NAND memory **6**.

[0348] When power loss occurs during execution of the fine program operation on the MG (WL N, SU1), the MG (WL N, SU1) transitions not to the foggy programmed state but to the program interrupted state. The information indicating the MG (WL N, SU1) has already been deleted from the foggy program management information **76**. Therefore, even if power loss occurs during execution of the fine program operation, the controller **5** reads the foggy program management information **76** from the NAND memory **6** at the restart of power supply to the memory system **3**, thereby correctly determining each storage location in the foggy programmed state (that is, the MG (WL N, SU1) is not in the foggy programmed state but in the program interrupted state) in a block allocated as a write destination block at the time of power loss.

[0349] On the basis of the foggy program management information **76**, therefore, the controller **5**

can correctly determine whether each storage location storing unreadable data is in the foggy programmed state or in the program interrupted state. The controller 5 is thus able to correctly determine whether a storage location in the program interrupted state is present in the block allocated as the write destination block at the time of the power loss. When determining that no storage location in the program interrupted state is present in the block, the controller 5 continues the data write operation on this block.

[0350] Specifically, when power supply to the memory system 3 is resumed, the controller 5 may execute the following unexpected power loss determination process.

[0351] The controller 5 reads the foggy program management information 76 from the NAND memory 6. By referring to the foggy program management information 76, the controller 5 checks foggy programmed states in the write destination block. Now a case is assumed where the foggy program management information 76 illustrated in FIG. 32 is stored in the NAND memory 6. The foggy program management information 76 illustrated in FIG. 32 indicates that the MG (WL N, SU2), the MG (WL N, SU3), the MG (WL N+1, SU0), and the MG (WL N+1, SU1) are storage locations in the foggy programmed state. The MG (WL N, SU2) is the oldest storage location among the storage locations in the foggy programmed state. The MG (WL N+1, SU1) is the latest storage location among the storage locations in the foggy programmed state.

[0352] The controller 5 determines whether a second condition that: (2-A) the storage location (MG (WL N, SU1)) on which the foggy program operation was executed immediately before execution of the foggy program operation on the oldest storage location (MG (WL N, SU2)) in the foggy programmed state, which is indicated by the foggy program management information 76, stores readable data; (2-B) each of the storage locations (MG (WL N, SU2), MG (WL N, SU3), MG (WL N+1, SU0), and MG (WL N+1, SU1)), which are indicated by the foggy program management information 76, stores unreadable data; and (2-C) the storage location (MG (WL N+1, SU2)) on which the foggy program operation is to be executed subsequently to execution of the foggy program operation on the latest storage location (MG (WL N+1, SU1)) in the foggy programmed state, which is indicated by the foggy program management information 76, is in the erased state, is satisfied.

[0353] When this condition is satisfied, the controller 5 determines that the previous write operation in the memory system 3 was stopped properly, and determines the MG (WL N+1, SU2) as the next write destination storage location.

[0354] When this condition is not satisfied, the controller 5 determines that the previous write operation in the memory system 3 was stopped improperly and a storage location in the program interrupted state is present in the write destination block, and then allocates a new block as the write destination block.

[0355] An operation of updating the foggy program management information 76 in response to completion of the foggy program operation, which is executed in a case where each block includes only one string unit, will then be described. FIG. 33 illustrates the foggy program operation in a case where each block includes only one string unit, which is executed in the memory system 3 according to the second embodiment.

[0356] In an initial state illustrated in FIG. 33, the MG (WL0) to MG (WL6) are each in the fine programmed state. The MG (WL7) is in the foggy programmed state. The foggy program management information 76, therefore, includes information indicating the MG (WL7).

[0357] Step 1: The controller 5 executes, for example, the foggy program operation on a storage location connected to the word line WL8.

[0358] Step 2: In response to completion of the foggy program operation, the controller 5 updates the foggy program management information 76 stored in the DRAM 7. The controller 5 adds information which indicates a storage location where the foggy program operation is completed, i.e., information indicating the MG (WL8), to the foggy program management information 76.

[0359] Step 3: The controller 5 writes the updated foggy program management information 76 into

the NAND memory 6.

[0360] An operation of updating the foggy program management information 76 before starting the fine program operation, which is executed in the case where each block includes only one string unit, will then be described. FIG. 34 illustrates the fine program operation in the case where each block includes only one string unit, which is executed in the memory system 3 according to the second embodiment. It is assumed in FIG. 34 that after the state illustrated in FIG. 33, the fine program operation on the MG (WL7) is started.

[0361] Step 1: Before starting the fine program operation on the MG (WL7), the controller 5 updates the foggy program management information 76. The controller 5 deletes information which indicates the MG (WL7) on which the fine program operation is to be executed next, from the foggy program management information 76.

[0362] Step 2: The controller 5 writes the updated foggy program management information 76 into the NAND memory 6.

[0363] Step 3: The controller 5 executes the fine program operation on the MG (WL7).

[0364] In this manner, even when each block includes only one string unit, the controller 5 executes updating of the foggy program management information 76 and writing of the foggy program management information 76 into the NAND memory 6 in the same manner as in the case of each block includes two or more string units.

[0365] A process of updating the foggy program management information 76 will then be described. FIG. 35 is a flowchart illustrating the procedure of a process of updating and storing the foggy program management information 76 in response to completion of the foggy program operation, which is executed in the memory system 3 according to the second embodiment.

[0366] First, the controller 5 determines whether the foggy program operation on a certain storage location has been completed (step S31).

[0367] When the foggy program operation is not completed yet (No in step S31), the controller 5 waits until the foggy program operation is completed.

[0368] When the foggy program operation has been completed (Yes in step S31), the controller 5 updates the foggy program management information 76 (step S32). The controller 5 adds, to the foggy program management information 76, information indicating the storage location that has transitioned to the foggy programmed state upon completion of the foggy program operation.

[0369] The controller 5 writes the foggy program management information 76 updated in step S32, into the NAND memory 6 (step S33). As a result, the updated foggy program management information 76 is stored in the NAND memory 6. The updated foggy program management information 76 may be written into, for example, an SLC block.

[0370] FIG. 36 is a flowchart illustrating the procedure of a process of updating and storing the foggy program management information 76 before starting the fine program operation, which is executed in the memory system 3 according to the second embodiment.

[0371] First, the controller 5 determines whether the fine program operation is executable on any one of the storage locations (step S41).

[0372] When the fine program operation is not executable (No in step S41), the controller 5 waits until the fine program operation becomes executable.

[0373] When the fine program operation is executable (Yes in step S41), the controller 5 updates the foggy program management information 76 (step S42). The controller 5 deletes, from the foggy program management information 76, information indicating a storage location on which the fine program operation, which has become executable, is to be executed.

[0374] The controller 5 writes the foggy program management information 76 updated in step S42, into the NAND memory 6 (step S43). As a result, the updated foggy program management information 76 is stored in the NAND memory 6.

[0375] The controller 5 then executes the fine program operation (step S44).

[0376] The unexpected power loss determination process will then be described. FIG. 37 is a

flowchart illustrating the procedure of the unexpected power loss determination process executed in the memory system **3** according to the second embodiment.

[0377] When power supply to the memory system **3** is resumed, the controller **5** reads the foggy program management information **76** from the NAND memory **6** (step S502).

[0378] The controller **5** acquires a foggy program status of a write destination block, from the foggy program management information **76** read in step S502 (step S503).

[0379] By referring to the foggy program management information **76**, the controller **5** checks the foggy program status of the write destination block (step S504). Specifically, the controller **5** identifies the oldest storage location in the foggy programmed state and the latest storage location in the foggy programmed state within the foggy program management information **76**. The controller **5** then determines whether a second condition that: (1) a storage location on which the foggy program operation was executed immediately before execution of the foggy program operation on the oldest storage location in the foggy programmed state, stores readable data; (2) each of storage locations which are indicated by the foggy program management information **76** stores unreadable data; and (3) a storage location on which the foggy program operation is to be executed subsequently to execution of the foggy program operation on the latest storage location in the foggy programmed state, is in the erased state, is satisfied.

[0380] When the second condition is satisfied (Yes in step S504), the controller **5** determines that the previous write operation in the memory system **3** was stopped properly, that is, no storage location in the program interrupted state is present in the block allocated as the write destination block (step S505).

[0381] On the basis of the foggy program management information **76** read in step S502, the controller **5** then determines the next write destination storage location (step S506). The storage location on which the foggy program operation is to be executed subsequently to the latest storage location in the foggy programmed state is thus determined as the next write destination storage location.

[0382] When the second condition is not satisfied (No in step S504), the controller **5** determines that the previous write operation in the memory system **3** was stopped improperly (step S507).

[0383] The controller **5** registers the block, which is allocated as the write destination block, in the block management table **72** as an unavailable block (step S508).

[0384] The controller **5** allocates another block as a new write destination block (step S509). The controller **5** selects the block from the free block list **73**, and allocates the selected block as the write destination block.

[0385] The controller **5** determines the head storage location of the write destination block as the next write destination storage location (step S510).

[0386] Basically, the controller **5** may execute the unexpected power loss determination process every time power supply is resumed, regardless of whether the previous power loss is unexpected power loss in which power supply is suddenly shut off or power supply is shut off after receiving a shutdown notice from the host **2**.

[0387] Note that the controller **5** may execute the unexpected power loss determination process in response to the restart of power supply only when the previous power loss was the unexpected power loss.

[0388] As described above, in the memory system **3** according to the second embodiment, the controller **5** can determine whether the previous write operation was stopped properly by using the foggy program management information **76** written in the NAND memory **6**, thereby correctly determining the next write destination storage location.

Third Embodiment

[0389] A memory system **3** according to the third embodiment has a configuration for reducing the number of times of write operations of writing the foggy program management information **76** into the NAND memory **6**.

[0390] Specifically, in the memory system **3** according to the third embodiment, the foggy program management information **76** is written into the NAND memory **6** only in the case of unexpected power loss in which power supply to the memory system **3** is shut off without notice from the host **2**.

[0391] A configuration of the memory system **3** according to the third embodiment will be described. FIG. **38** illustrates an example of a configuration of the information processing system **1** that includes the memory system **3** according to the third embodiment.

[0392] Most of the components included in the memory system **3** according to the third embodiment are the same as those included in the memory system **3** according to the second embodiment illustrated in FIG. **30**. The following description, therefore, will be focused on differences between the memory system **3** according to the third embodiment and the memory system **3** according to the second embodiment.

[0393] The memory system **3** according to the third embodiment further includes the capacitor **81**.

[0394] When unexpected power loss has occurred, the controller **5** executes a process of storing the foggy program management information **76** to the NAND memory **6** by using power from the capacitor **81**.

[0395] The foggy program operation and an operation of writing the foggy program management information **76** into the NAND memory **6** will then be described. FIG. **39** illustrates the foggy program operation and a process of storing the foggy program management information **76** to the NAND memory **6**, which are executed in the memory system **3** according to the third embodiment.

[0396] Step **1**: The controller **5** executes, for example, the foggy program operation on the MG (WL N+1, SU1).

[0397] Step **2**: In response to completion of the foggy program operation, the controller **5** updates the foggy program management information **76** stored in the DRAM **7**. The controller **5** adds information which indicates the MG (WL N+1, SU1) where the foggy program operation has been completed, to the foggy program management information **76**.

[0398] In the same manner as in the second embodiment, before starting the fine program operation on a certain storage location, the controller **5** updates the foggy program management information **76** stored in the DRAM **7** so as to delete information indicative of the certain storage location.

[0399] When having detected occurrence of unexpected power loss, the controller **5** writes the foggy program management information **76** stored in the DRAM **7**, into the NAND memory **6**. At this time, the controller **5** and the NAND memory **6** operate by using power accumulated in the capacitor **81**. After writing the foggy program management information **76** into the NAND memory **6**, the controller **5** controls the power supply circuit **8** to stop operating of the memory system **3**.

[0400] Then, when power supply to the memory system **3** is resumed, the controller **5** reads the foggy program management information **76** from the NAND memory **6**.

[0401] A process of managing the foggy program management information **76** will then be described. FIG. **40** is a flowchart illustrating the procedure of a process of managing the foggy program management information **76**, which is executed in the memory system **3** according to the third embodiment.

[0402] First, the controller **5** determines whether unexpected power loss has occurred (step **S61**).

[0403] When unexpected power loss has occurred (Yes in step **S61**), the controller **5** writes the foggy program management information **76** stored in the DRAM **7**, into the NAND memory **6** (step **S62**).

[0404] When unexpected power loss has not occurred (No in step **S61**), the controller **5** determines whether the foggy program operation is executable (step **S63**).

[0405] When the foggy program operation is not executable (No in step **S63**), the controller **5** waits until the foggy program operation becomes executable or unexpected power loss occurs.

[0406] When the foggy program operation is executable (Yes in step **S63**), the controller **5** executes the foggy program operation (step **S64**).

[0407] In response to completion of the foggy program operation, the controller 5 then updates the foggy program management information 76 (step S65). The controller 5 adds information which indicates a storage location where the foggy program operation has been completed, to the foggy program management information 76.

[0408] The controller 5 determines whether the fine program operation is executable (step S66).

[0409] When the fine program operation is not executable (No in step S66), the controller 5 waits until the fine program operation becomes executable.

[0410] When the fine program operation is executable (Yes in step S66), the controller 5 updates the foggy program management information 76 before starting the fine program operation (step S67). The controller 5 deletes information which indicates a storage location where the fine program operation is to be executed, from the foggy program management information 76.

[0411] The controller 5 then executes the fine program operation (step S68). Subsequently, the process by the controller 5 returns to step S61.

[0412] Through these steps, in the memory system 3 according to the third embodiment, the controller 5 can determine whether the previous write operation was stopped properly, by the same method as executed in the second embodiment.

[0413] In addition, by using the capacitor 81, the foggy program management information 76 is written into the NAND memory 6 only when unexpected power loss occurred. As a result, the frequency of operations of writing the foggy program management information 76 into the NAND memory 6 can be reduced as compared to a case where the foggy program management information 76 is written into the NAND memory 6 every time the foggy program management information 76 is updated. This suppresses effects on write amplification and I/O performance.

Fourth Embodiment

[0414] Components of a memory system 3 according to a fourth embodiment are the same as the components of the memory system 3 according to the third embodiment that have been described with reference to FIG. 38, and therefore will be omitted in further description.

[0415] A data write operation executed in the memory system 3 according to the fourth embodiment will then be described. In the memory system 3 according to the fourth embodiment, when unexpected power loss has occurred, a write operation of transitioning every storage location in the foggy programmed state to the fine programmed state is executed by using power accumulated in the capacitor 81. FIG. 41 illustrates an additional write operation of transitioning the threshold voltage of a memory cell from the foggy programmed state to the fine programmed state, which is executed in the memory system 3 according to the fourth embodiment.

[0416] As illustrated in FIG. 41 (a), the threshold voltage of each of a plurality of memory cells included in a storage location in the foggy programmed state, where the foggy program operation has been executed but the fine program operation has not been executed yet, is set with low precision in which data cannot be read correctly.

[0417] In the memory system 3 according to the fourth embodiment, in a case where unexpected power loss occurs when a storage location in the foggy programmed state is present, the controller 5 executes the fine program operation on this storage location by using power stored in the capacitor 81.

[0418] As illustrated in FIG. 41 (b), each of a plurality of memory cells included in a storage location in the fine programmed state has a threshold voltage corresponding to a data value to be stored in each memory cell.

[0419] A block where a data write operation is executed at occurrence of unexpected power loss will then be described. FIG. 42 illustrates a state of storage locations of a block in which unexpected power loss has occurred and a state of the storage locations of the block after execution of the additional write operation, which are in the memory system 3 according to the fourth embodiment.

[0420] FIG. 42 (a) illustrates the state of storage locations of a block in which unexpected power

loss has occurred. It is assumed in FIG. 42 (a) that unexpected power loss has occurred in a period between completion of the foggy program operation on the MG (WL N+2, SU2) and completion of the fine program operation on the MG (WL N+1, SU2).

[0421] When unexpected power loss has occurred, the controller 5 executes the additional write operation of transitioning each storage location in the foggy programmed state that is included in a write destination block, to the fine programmed state by using power accumulated in the capacitor 81.

[0422] To execute the fine program operation on a storage location included in each string unit connected to a certain word line, the foggy program operation needs to be executed on a storage location that is connected to the next word line subsequent to the certain word line and that is included in the same string unit. In an example of the additional write operation illustrated in FIG. 42 (a), the controller 5 completes the fine program operation on the MG (WL N+1, SU2), and then executes the foggy program operation on the MG (WL N+2, SU3). In this case, dummy data may be written into the MG (WL N+2, SU3).

[0423] FIG. 42 (b) illustrates a state of storage locations of the write destination block after the additional write operation is executed. As illustrated in FIG. 42 (b), all the storage locations included in the write destination block are in the fine programmed state. Because the write destination block becomes full, the controller 5 registers the write destination block in the QLC active block list 75.

[0424] Then, when power supply to the memory system 3 is resumed, the controller 5 selects another block from the free block list 73, and allocates the selected block as a new write destination block. The controller 5 then determines the head storage location of the new write destination block as the next write destination storage location.

[0425] The procedure of a data write operation executed at occurrence of unexpected power loss will then be described. FIG. 43 is a flowchart illustrating the procedure of a data write operation executed at occurrence of unexpected power loss, in the memory system 3 according to the fourth embodiment.

[0426] First, the controller 5 determines whether unexpected power loss has occurred (step S71).

[0427] When unexpected power loss has not occurred (No in step S71), the controller 5 waits until unexpected power loss occurs.

[0428] When unexpected power loss has occurred (Yes in step S71), the controller 5 executes the additional write operation (step S72). The controller 5 continues executing the additional write operation until the fine program operations on all storage locations included in a write destination block are completed, that is, until the write destination block becomes full.

[0429] In this manner, according to the memory system 3 of the fourth embodiment, when unexpected power loss occurs, the controller 5 continues executing the additional write operation until the write destination block becomes full. Because of this, when power supply to the memory system 3 is resumed, the controller 5 does not need to determine whether the previous power loss was unexpected power loss.

Fifth Embodiment

[0430] Components of a memory system 3 according to a fifth embodiment are the same as the components of the memory system 3 according to the first embodiment that have been described with reference to FIG. 1, and therefore will be omitted in further description.

[0431] In the fifth embodiment, the controller 5 writes a flag (foggy flag) to at least one memory cell for storing management information among a plurality of memory cells included in a storage location. The foggy flag is information indicating that the storage location is in the foggy programmed state.

[0432] The foggy flag will first be described. FIG. 44 (a) illustrates threshold voltage distributions of a plurality of memory cells included in a storage location in the foggy programmed state, in the memory system 3 according to the fifth embodiment. FIG. 44 (b) illustrates threshold voltage

distributions of a plurality of memory cells included in a storage location in the program interrupted state in which power loss occurred during execution of the foggy program operation, in the memory system **3** according to the fifth embodiment.

[0433] Each of storage locations included in a block includes a plurality of memory cells for storing data and at least one memory cell for storing the management information. Hereinafter, the at least one memory cell for storing the management information is referred to as a redundant memory cell.

[0434] In response to completion of the foggy program operation on a certain storage location, the controller **5** writes a foggy flag to the redundant memory cell included in the certain storage location.

[0435] The threshold voltage of the redundant memory cell to which the foggy flag has been written is higher than a threshold voltage range used in the foggy-fine program operation. Specifically, in response to completion of the foggy program operation on a certain storage location, the controller **5** sets the threshold voltage of the redundant memory cell included in the certain storage location, to a threshold voltage higher than the threshold voltage range used in the foggy-fine program operation, thereby writing the foggy flag to the redundant memory cell.

[0436] As illustrated in FIG. **44** (a), the threshold voltage distributions of the memory cells included in the storage location where the foggy program operation is completed are set with low precision in which data cannot be read correctly. The threshold voltage of the redundant memory cell among the memory cells is set higher than the threshold voltage range used in the foggy-fine program operation. Specifically, the threshold voltage of the redundant memory cell to which the foggy flag has been written is set higher than a foggy flag threshold voltage and lower than a fine flag threshold voltage. The foggy flag threshold voltage is higher than, for example, V_{r16} , which has been described with reference to FIG. **6**.

[0437] The controller **5** can determine that the foggy flag has been written to the storage location by, for example, confirming that the storage location includes a memory cell having a threshold voltage equal to or higher than the foggy flag threshold voltage but does not include a memory cell having a threshold voltage equal to or higher than the fine flag threshold voltage.

[0438] In this manner, the storage location where the foggy program operation is completed stores unreadable data and further stores the foggy flag. The controller **5** is thus able to determine that the storage location in which the foggy flag is present and from which data cannot be read correctly is a storage location in the foggy programmed state.

[0439] As illustrated in FIG. **44** (b), no foggy flag is written to the redundant memory cell of a storage location in the program interrupted state. Each of threshold voltages of all memory cells included in the storage location are, therefore, within the threshold voltage range used in the foggy-fine program operation.

[0440] The controller **5** can determine that the foggy flag is not written in the storage location by, for example, confirming that the storage location includes no memory cell having a threshold voltage equal to or higher than the foggy flag threshold voltage.

[0441] In this manner, the storage location where power loss occurred during execution of the foggy program operation stores unreadable data but does not store the foggy flag. The controller **5** is thus able to determine that the storage location in which the foggy flag is not present and from which data cannot be read correctly is a storage location in the program interrupted state, that is, the storage location where power loss occurred during execution of the foggy program operation.

[0442] A fine flag will then be described. FIG. **45** (a) illustrates threshold voltage distributions of a plurality of memory cells included in a storage location in the program interrupted state, in which power loss occurred during execution of the fine program operation, in the memory system **3** according to the fifth embodiment. FIG. **45** (b) illustrates threshold voltage distributions of a plurality of memory cells included in a storage location in the fine programmed state, in the memory system **3** according to the fifth embodiment.

[0443] When the fine program operation on a certain storage location becomes executable, the controller 5 removes a foggy flag from the redundant memory cell included in the certain storage location before starting the fine program operation. Specifically, the controller 5 changes the threshold voltage of the redundant memory cell from the threshold voltage corresponding to the foggy flag threshold voltage to the threshold voltage corresponding to the fine flag threshold voltage, thereby removing the foggy flag from the redundant memory cell. The redundant memory cell having the threshold voltage corresponding to the fine flag threshold voltage is also referred to as a memory cell in which the fine flag is written. In other words, the controller 5 writes the fine flag to the redundant memory cell, thereby removing the foggy flag from the redundant memory cell.

[0444] As illustrated in FIG. 45 (a), the foggy flag is not written in the redundant memory cell of a storage location in the program interrupted state. Specifically, the storage location where power loss occurred during execution of the fine program operation stores unreadable data but does not store the foggy flag (and stores the fine flag instead). The controller 5 is thus able to determine that the storage location in which the foggy flag is not present and from which data cannot be read correctly is a storage location in the program interrupted state.

[0445] The controller 5 may determine whether the fine flag is present in the storage location. In this case, the controller 5 can also determine that the storage location in which the fine flag is present and from which data cannot be read correctly is a storage location where power loss occurred during execution of the fine program operation.

[0446] As illustrated in FIG. 45 (b), the foggy flag is not written in the redundant memory cell of a storage location in the fine programmed state. Specifically, the storage location where the fine program operation is completed stores readable data but does not store the foggy flag (and stores the fine flag instead). The controller 5 thus determines that the storage location from which data can be read correctly is a storage location in the fine programmed state.

[0447] In this manner, by checking the foggy flag of a storage location that stores unreadable data, the controller 5 can correctly determine that the storage location is in the foggy programmed state. Specifically, depending on whether the foggy flag is stored in each storage location storing unreadable data, the controller 5 can correctly determine whether each storage location storing the unreadable data is in the foggy programmed state or in the program interrupted state. The controller 5 is thus able to correctly determine whether a storage location in the program interrupted state is present in the block that is allocated as the write destination block at the time of the power loss. When determining that no storage location in the program interrupted state is present in the block, the controller 5 continues the data write operation on this block.

[0448] Specifically, when power supply to the memory system 3 is resumed, the controller 5 may execute the following unexpected power loss determination process.

[0449] When power supply to the memory system 3 has been resumed after power supply to the memory system 3 was shut off, the controller 5 identifies a storage location storing data that becomes readable last among a plurality of storage locations included in the write destination block. The controller 5 then determines whether a third condition that: (3-A) each of a first number of storage locations on which the foggy program operation was executed following execution of the foggy program operation on the storage location storing data that becomes readable last, stores unreadable data and the foggy flag; and (3-B) a storage location on which the foggy program operation is to be executed following execution of the foggy program operation on the first number of the storage locations, is in the erased state, is satisfied.

[0450] The first number is determined on the basis of the number of string units SU included in each block.

[0451] For example, in a case where each block includes four string units SU, the first number is four (or five). In a case where each block includes only one string unit SU, the first number is one (or two).

[0452] In the case where each block includes four string units SU, therefore, the controller 5 determines whether the third condition that: each of four or five storage locations on which the foggy program operation is to be executed following execution of the foggy program operation on the storage location storing data that becomes readable last, stores unreadable data and the foggy flag; and a storage location on which the foggy program operation is to be executed subsequently to execution of the foggy program operation on the four or five storage locations, is in the erased state, is satisfied.

[0453] For example, as in the example illustrated in FIG. 7, at the point of completion of the fine program operation on the MG (WL1, SU2), the MG (WL1, SU2) is a storage location storing data that becomes readable last. In this case, the storage location on which the foggy program operation has been executed subsequently to execution of the foggy program operation on the MG (WL1, SU2) is the MG (WL1, SU3). The controller 5 determines whether the MG (WL1, SU3) stores unreadable data and the foggy flag. The controller 5 further determines whether three storage locations, i.e., the MG (WL2, SU0) to the MG (WL2, SU2) each store unreadable data and the foggy flag. In this case, the first number is four. The controller 5 then determines whether the MG (WL2, SU3), which is a storage location on which the foggy program operation is to be executed subsequently to execution of the foggy program operation on the MG (WL2, SU2), is in the erased state. Through this process, the controller 5 determines whether the third condition is satisfied.

[0454] In the case where each block includes only one string unit SU, the controller 5 determines whether a condition is satisfied, the condition requiring that each of one or two storage locations on which the foggy program operation is to be executed subsequently to execution of the foggy program operation on a storage location that is the last to store readable data, stores unreadable data and the foggy flag, and that a storage location on which the foggy program operation is to be executed subsequently to execution of the foggy program operation on the one or two storage locations, is in the erased state.

[0455] Each storage location that stores unreadable data and the foggy flag is a storage location in the foggy programmed state in which the foggy program operation is completed and the fine program operation is not started yet.

[0456] When this condition is satisfied, therefore, the controller 5 can determine that no storage location in the program interrupted state is present in the block that is allocated as the write destination block. In this case, the controller 5 determines, as the next write destination storage location, the storage location in the erased state on which the foggy program operation is to be executed subsequently to execution of the foggy program operation on the storage location that stores unreadable data and the foggy flag.

[0457] The foggy program operation will then be described. FIG. 46 is a flowchart illustrating the procedure of the foggy program operation executed in the memory system 3 according to the fifth embodiment.

[0458] First, the controller 5 determines whether the foggy program operation on a storage location where the foggy program operation is to be executed next, is executable (step S81).

[0459] When the foggy program operation is not executable (No in step S81), the controller 5 waits until the foggy program operation becomes executable.

[0460] When the foggy program operation is executable (Yes in step S81), the controller 5 executes the foggy program operation on the storage location (step S82).

[0461] When the foggy program operation on the storage location is completed, the controller 5 sets the threshold voltage of the redundant memory cell of the storage location to a voltage that corresponds to the foggy flag threshold voltage, thereby writing the foggy flag to the redundant memory cell (step S83).

[0462] The fine program operation will then be described. FIG. 47 is a flowchart illustrating the procedure of the fine program operation executed in the memory system 3 according to the fifth embodiment.

[0463] First, the controller 5 determines whether the fine program operation on a storage location where the fine program operation is to be executed next, is executable (step S91).

[0464] When the fine program operation is not executable (No in step S91), the controller 5 waits until the fine program operation becomes executable.

[0465] When the fine program operation is executable (Yes in step S91), the controller 5 removes the foggy flag from the redundant memory cell of the storage location before starting the fine program operation on the storage location (step S92). In other words, the controller 5 changes the threshold voltage of the redundant memory cell to a voltage that corresponds to the fine flag threshold voltage.

[0466] The controller 5 then executes the fine program operation on the storage location (step S93).

[0467] The procedure of the unexpected power loss determination process will then be described.

FIG. 48 is a flowchart illustrating the procedure of the unexpected power loss determination process executed in the memory system 3 according to the fifth embodiment.

[0468] When power supply to the memory system 3 is resumed, the controller 5 acquires information indicative of a write destination storage location, from the NAND memory 6 (step S1002).

[0469] Subsequently, the controller 5 executes data read operations on several storage locations that start from the storage location indicated by the acquired information, thereby identifying the last storage location that stores readable data (steps S1003 to S1005).

[0470] In this case, the controller 5 first specifies the storage location indicated by the acquired information, as a storage location on which the data read operation is to be executed first (step S1003).

[0471] The controller 5 determines whether data written in the specified storage location is readable data (step S1004). When data written in the specified storage location is readable data (Yes in step S1004), the controller 5 specifies the next storage location subsequent to the specified storage location (step S1005). Subsequently, the process by the controller 5 returns to step S1004. In step S1004, the controller 5 determines whether the data written in the storage location specified in step S1005 is readable data.

[0472] When a storage location subsequent to the storage location storing the readable data stores unreadable data, the controller 5 determines the storage location storing the readable data, as the last storage location that stores readable data.

[0473] When determining the last storage location that stores readable data, the controller 5 determines whether the storage location subsequent to the last storage location storing readable data (that is, the storage location on which the foggy program operation has been executed subsequently to execution of the foggy program operation on the last storage location), is in the foggy programmed state (steps S1006 to S1009).

[0474] When the subsequent storage location stores unreadable data and stores the foggy flag as well, the controller 5 can determine that the subsequent storage location is in the foggy programmed state. Note that it has already been confirmed in step S1004 that the subsequent storage location stores unreadable data. The controller 5, therefore, just determines whether the subsequent storage location stores the foggy flag.

[0475] Alternatively, the controller 5 may first determine whether the subsequent storage location is in the erased state. In this case, when determining that the subsequent storage location is not in the erased state, the controller 5 may determine whether the subsequent storage location stores the foggy flag.

[0476] Here, a case will be described where the controller 5 first determines whether the subsequent storage location is in the erased state, and when determining that the subsequent storage location is not in the erased state, then determines whether the subsequent storage location stores the foggy flag.

[0477] The controller 5 determines whether the subsequent storage location is in the erased state

(step **S1006**). In other words, the controller **5** determines whether the subsequent storage location includes a memory cell that has a threshold voltage equal to or higher than the erased page threshold voltage.

[0478] When the subsequent storage location is not in the erased state (No in step **S1006**), the controller **5** determines whether a flag is written in the subsequent storage location (step **S1007**). The controller **5** determines whether a plurality of memory cells included in the subsequent storage location include a memory cell that has a threshold voltage equal to or higher than the foggy flag threshold voltage.

[0479] When the flag is written in the subsequent storage location (Yes in step **S1007**), the controller **5** determines whether the flag written in the subsequent storage location is the foggy flag or the fine flag (step **S1008**). The controller **5** determines whether the memory cells included in the subsequent storage location include a memory cell that has a threshold voltage equal to or higher than the fine flag threshold voltage. When the memory cells include a memory cell that has a threshold voltage equal to or higher than the fine flag threshold voltage, the controller **5** determines that the flag written in the subsequent storage location is the fine flag. When the memory cells does not include a memory cell that has a threshold voltage equal to or higher than the fine flag threshold voltage, the controller **5** determines that the flag written in the subsequent storage location is the foggy flag.

[0480] When the flag written in the subsequent storage location is the foggy flag (foggy flag in step **S1008**), the controller **5** specifies a storage location further subsequent to the subsequent storage location (step **S1009**). The process by the controller **5** then returns to step **S1006**. In step **S1006**, the controller **5** determines whether the storage location specified in step **S1009** is in the erased state.

[0481] In this manner, the controller **5** determines whether each of the first number of the storage locations on which the foggy program operation has been executed subsequently to execution of the foggy program operation on the storage location storing data that becomes readable last, stores unreadable data and the foggy flag.

[0482] A storage location subsequent to the first number of storage locations that each store unreadable data and the foggy flag, is specified in step **S1009**. Subsequently, in step **S1006**, the controller **5** determines whether the storage location subsequent to the first number of the storage locations is in the erased state.

[0483] When the subsequent storage location is in the erased state (Yes in step **S1006**), the controller **5** determines that the previous write operation in the memory system **3** was stopped properly, that is, no storage location in the program interrupted state is present in the block that is allocated as the write destination block (step **S1010**).

[0484] The controller **5** then determines the subsequent storage location as the next write destination storage location (step **S1011**).

[0485] When the subsequent storage location is not in the erased state (No in step **S1006**) and no foggy flag is written in the subsequent storage location (No in step **S1007** or fine flag in step **S1008**), the controller **5** determines that the previous write operation was stopped improperly (step **S1012**).

[0486] Furthermore, in the process of determining whether the first number of storage locations each store unreadable data and the foggy flag, when no foggy flag is written in the target storage location (No in step **S1007** or fine flag in step **S1008**), the controller **5** determines that the previous write operation was stopped improperly (step **S1012**). In other words, the controller **5** determines that there is a possibility that a storage location in the program interrupted state is present in the block that is allocated as the write destination block.

[0487] The controller **5** thus determines the write destination block as an unavailable block (step **S1013**).

[0488] The controller **5** allocates another block, as a new write destination block (step **S1014**). The controller **5** selects the block from the free block list **73**. The controller **5** then allocates the selected

block as a write destination block.

[0489] The controller 5 determines the head storage location of the write destination block, as the next write destination storage location (step S1015). Specifically, the controller 5 determines the head storage location of the block allocated as the write destination block in step S1014, as the next write destination storage location.

[0490] As described above, in the fifth embodiment, the foggy flag is written to at least one memory cell (redundant memory cell) among a plurality of memory cells included in each storage location. Then, in response to resuming of power supply to the memory system 3 after shut off of power supply to the memory system 3, the controller 5 determines whether a condition that: each of the first number of the storage locations on which the foggy program operation has been executed subsequently to execution of the foggy program operation on the storage location storing data that becomes readable last, stores unreadable data and the foggy flag; and a storage location on which the foggy program operation is to be executed subsequently to execution of the foggy program operations on the first number of the storage locations is in the erased state, is satisfied. This enables the controller 5 to correctly determine whether the previous write operation was stopped properly.

[0491] Basically, the controller 5 may execute the unexpected power loss determination process every time power supply is resumed, regardless of whether the previous power loss was unexpected power loss in which power supply is suddenly shut off or power supply was shut off after receiving a shutdown notice from the host 2.

[0492] Note that the controller 5 may execute the unexpected power loss determination process in response to the restart of power supply only when the previous power loss was the unexpected power loss.

[0493] As described above, according to the first to fifth embodiments, even when power supply to the memory system 3 is suddenly shut off due to unexpected power loss during execution of the first step program operation (foggy program operation) or the second step program operation (foggy program operation), the next write destination storage location can be correctly determined.

[0494] It should be noted that any two or more of the first to fifth embodiments may be combined appropriately.

[0495] In the first to fifth embodiments, the case where data received from the host 2 is written into an SLC block and then the data stored in the SLC block is written into a QLC block by using a multi-step program operation, has been described. However, the first to fifth embodiments are not limited to this configuration. For example, data received from the host 2 may be written directly into the QLC block by using the multi-step program operation.

[0496] The last storage location that stores readable data is a storage location where the second step program operation (i.e., fine program operation) has been executed last. Unreadable data is also referred to as data impossible of reading.

[0497] While certain embodiments have been described, these embodiments have been presented by way of example only, and are not intended to limit the scope of the inventions. Indeed, the novel devices and methods described herein may be embodied in a variety of other forms; furthermore, various omissions, substitutions and changes in the form of the embodiments described herein may be made without departing from the spirit of the inventions. The accompanying claims and their equivalents are intended to cover such forms or modification as would fall within the scope and spirit of the inventions.

Claims

1. A memory system comprising: a nonvolatile memory that includes a plurality of blocks, each of the plurality of blocks being a unit of a data erase operation and including a plurality of storage locations, each of the plurality of storage locations including a plurality of memory cells; and a

controller electrically connected to the nonvolatile memory and configured to execute a data write operation on a write destination block among the plurality of blocks by using a multi-step program operation that includes at least a first step program operation and a second step program operation, wherein the controller is configured to execute the data write operation on the plurality of storage locations of the write destination block in an order in which: the first step program operation on a first storage location of the write destination block is executed; after completion of the first step program operation on the first storage location, the first step program operation on a second storage location of the write destination block is executed; in response to completion of the first step program operation on the second storage location, the second step program operation on the first storage location is executed; and after completion of the second step program operation on the first storage location, the first step program operation on a third storage location of the write destination block is executed, wherein the controller is further configured to: in executing the data write operation on the write destination block according to the order, manage first information indicating whether each of one or more storage locations of the plurality of storage locations of the write destination block is in a first state in which the first step program operation is completed and the second step program operation is not started yet by: updating, in response to completion of the first step program operation on a storage location of the write destination block, the first information so that information indicating that the first step program operation has been completed on the storage location is added to the first information; and updating, before starting the second step program operation on a storage location of the write destination block, the first information so that information indicating that the second step program operation is to be started on the storage location is deleted from the first information; and store the updated first information to the nonvolatile memory.

2. The memory system according to claim 1, wherein the controller is further configured to, upon resuming of power supply to the memory system after shutoff of power supply to the memory system: read the first information from the nonvolatile memory; determine, for a first block among the plurality of blocks, whether a second condition that (A) a fourth storage location of the first block stores readable data, (B) each storage location of the first block indicated by the first information stores unreadable data, and (C) each of the plurality of memory cells of a fifth storage location of the first block has a threshold voltage corresponding to an erased state, is satisfied, wherein, among the plurality of storage locations of the first block, according to the order, the first step program operation on the fourth storage location has been executed; after completion of the first step program operation on the fourth storage location, the first step program operation on the oldest storage location of the storage locations that are indicated by the first information to be in the first state has been executed; in response to completion of the first step program operation on the latest storage location of the storage locations that are indicated by the first information to be in the first state, the second step program operation on the fourth storage location has been executed; and the first step program operation on the fifth storage location is to be executed after completion of the second step program operation on the fourth storage location, and when the second condition is satisfied, determine the fifth storage location of the first block as a next write destination storage location.

3. The memory system according to claim 2, wherein the controller is further configured to, when the second condition is not satisfied: manage the first block as an unavailable block; allocate a second block among the plurality of blocks as a new write destination block; and determine a head storage location of the plurality of storage locations of the second block, as the next write destination storage location.

4. The memory system according to claim 2, wherein the controller is further configured to, upon resuming of power supply to the memory system after power supply to the memory system is shut off by unexpected power loss: read the first information from the nonvolatile memory; determine whether the second condition is satisfied; and when the second condition is satisfied, determine the

fifth storage location of the first block as the next write destination storage location.

5. The memory system according to claim 1, further comprising: a capacitor configured to store power supplied to the memory system from an external power source, wherein the controller is further configured to: not store the updated first information to the nonvolatile memory while power is supplied to the memory system from the external power source; and store the updated first information to the nonvolatile memory by using power stored in the capacitor when power supply to the memory system from the external power source is shut off.

6. The memory system according to claim 2, wherein the controller is further configured to: store write destination information about a write destination storage location to the nonvolatile memory; and upon resuming of power supply to the memory system after shutoff of power supply to the memory system, identify the fourth storage location by reading data from one or more of the plurality of storage locations of the first block from a last storage location of the first block indicated by the write destination information.

7. The memory system according to claim 2, wherein the first step program operation makes a threshold voltage of at least one of the plurality of memory cells of a write destination storage location higher than a threshold voltage that corresponds to the erased state, but data written to the write destination storage location is unreadable after completion of the first step operation, the second step program operation adjusts the threshold voltage of the at least one of the plurality of memory cells of the write destination storage location, and data written to the write destination storage location becomes readable after completion of the second step program operation.

8. The memory system according to claim 1, wherein the nonvolatile memory is a NAND flash memory.

9. A controller configured to control a nonvolatile memory, the nonvolatile memory including a plurality of blocks, each of the plurality of blocks being a unit of a data erase operation and including a plurality of storage locations, each of the plurality of storage locations including a plurality of memory cells, the controller is configured to execute a data write operation on a write destination block among the plurality of blocks by using a multi-step program operation that includes at least a first step program operation and a second step program operation, wherein the controller is configured to execute the data write operation on the plurality of storage locations of the write destination block in an order in which: the first step program operation on a first storage location of the write destination block is executed; after completion of the first step program operation on the first storage location, the first step program operation on a second storage location of the write destination block is executed; in response to completion of the first step program operation on the second storage location, the second step program operation on the first storage location is executed; and after completion of the second step program operation on the first storage location, the first step program operation on a third storage location of the write destination block is executed, wherein the controller is further configured to: in executing the data write operation on the write destination block according to the order, manage first information indicating whether each of one or more storage locations of the plurality of storage locations of the write destination block is in a first state in which the first step program operation is completed and the second step program operation is not started yet by: updating, in response to completion of the first step program operation on a storage location of the write destination block, the first information so that information indicating that the first step program operation has been completed on the storage location is added to the first information; and updating, before starting the second step program operation on a storage location of the write destination block, the first information so that information indicating that the second step program operation is to be started on the storage location is deleted from the first information; and store the updated first information to the nonvolatile memory.

10. The controller according to claim 9, wherein the controller is further configured to, upon resuming of power supply to a memory system that includes the controller and the nonvolatile

memory after shutoff of power supply to the memory system: read the first information from the nonvolatile memory; determine, for a first block among the plurality of blocks, whether a second condition that (A) a fourth storage location of the first block stores readable data, (B) each storage location of the first block indicated by the first information stores unreadable data, and (C) each of the plurality of memory cells of a fifth storage location of the first block has a threshold voltage corresponding to an erased state, is satisfied, wherein, among the plurality of storage locations of the first block, according to the order, the first step program operation on the fourth storage location has been executed; after completion of the first step program operation on the fourth storage location, the first step program operation on the oldest storage location of the storage locations that are indicated by the first information to be in the first state has been executed; in response to completion of the first step program operation on the latest storage location of the storage locations that are indicated by the first information to be in the first state, the second step program operation on the fourth storage location has been executed; and the first step program operation on the fifth storage location is to be executed after completion of the second step program operation on the fourth storage location, and when the second condition is satisfied, determine the fifth storage location of the first block as a next write destination storage location.

11. The controller according to claim 10, wherein the controller is further configured to, when the second condition is not satisfied: manage the first block as an unavailable block; allocate a second block among the plurality of blocks as a new write destination block; and determine a head storage location of the plurality of storage locations of the second block, as the next write destination storage location.

12. The controller according to claim 10, wherein the controller is further configured to, upon resuming of power supply to the memory system after power supply to the memory system is shut off by unexpected power loss: read the first information from the nonvolatile memory; determine whether the second condition is satisfied; and when the second condition is satisfied, determine the fifth storage location of the first block as the next write destination storage location.

13. The controller according to claim 9, wherein the controller, the nonvolatile memory, and a capacitor are included in a memory system, the capacitor being configured to store power supplied to the memory system from an external power source, wherein the controller is further configured to: not store the updated first information to the nonvolatile memory while power is supplied to the memory system from the external power source; and store the updated first information to the nonvolatile memory by using power stored in the capacitor when power supply to the memory system from the external power source is shut off.

14. The controller according to claim 10, wherein the controller is further configured to: store write destination information about a write destination storage location to the nonvolatile memory; and upon resuming of power supply to the memory system after shutoff of power supply to the memory system, identify the fourth storage location by reading data from one or more of the plurality of storage locations of the first block from a last storage location of the first block indicated by the write destination information.

15. The controller according to claim 10, wherein the first step program operation makes a threshold voltage of at least one of the plurality of memory cells of a write destination storage location higher than a threshold voltage that corresponds to the erased state, but data written to the write destination storage location is unreadable after completion of the first step operation, the second step program operation adjusts the threshold voltage of the at least one of the plurality of memory cells of the write destination storage location, and data written to the write destination storage location becomes readable after completion of the second step program operation.

16. The controller according to claim 9, wherein the nonvolatile memory is a NAND flash memory.
